

MASTERARBEIT | MASTER'S THESIS

Titel | Title

Consistent Chronologies: Evaluating Logical Constraints for
Temporal Relation Extraction in Text

verfasst von | submitted by
Maximilian Moser BA

angestrebter akademischer Grad | in partial fulfilment of the requirements for the degree of
Master of Arts (MA)

Wien | Vienna, 2026

Studienkennzahl lt. Studienblatt | Degree
programme code as it appears on the
student record sheet:

UA 066 647

Studienrichtung lt. Studienblatt | Degree
programme as it appears on the student
record sheet:

Masterstudium Digital Humanities

Betreut von | Supervisor:

Univ.-Prof. Dr. -Ing. Benjamin Roth B.Sc. M.Sc.

Contents

1	Introduction	11
2	The Evolution of Temporal Relation Extraction and Temporal Reasoning	15
2.1	Early Linguistic and Rule-Based Foundations	15
2.2	From Annotation Standardization to Supervised Learning	16
2.3	From Local Classifications to Temporal Reasoning	17
2.4	The Neural Turn	19
2.5	The Rise of the Transformers	19
2.6	Can Large Language Models Perform Temporal Reasoning?	21
3	Methodology	23
3.1	Constraints	23
3.2	Objective and Training Algorithm	27
3.3	Architecture	29
3.4	Data Augmentation	30
4	DomiKnowS Implementation	35
4.1	The DomiKnowS Api	35
4.2	Implementing Temporal Constraints with DomiKnowS	36
4.3	Integration Testing of Constraints with DomiKnowS	38
5	Experimental Setup	41
5.1	Data and Splits	41
5.2	Evaluation Metrics	42
5.3	Baselines	43
5.4	Hyperparameters and Training Details	43
6	Experiments	45
6.1	Main Results	45
6.2	Relation-Specific Performance	47
6.3	Constraint Satisfaction	49
6.4	Constraints Ablation Study	50
6.4.1	Relation-Specific Constraint Analysis	50
6.4.2	Inverse vs Transitive Constraints	53

6.5	Constraint Effects on Reduced Training Data	54
7	Discussion	57
8	Conclusion	61

Acknowledgement

I would like to thank Prof. Benjamin Roth and Vasiliki Kougia for their excellent supervision of this thesis. Many thanks also to my mentors Renato and Ouassim, who played a big role in my transition from a historian to an AI engineer. I would also like to thank my family, who continuously supported me throughout my studies.

Lastly, I want to acknowledge the use of Large Language Models for text editing purposes to improve grammar and style and for support in generating figures.

Zusammenfassung

Die Extraktion temporaler Relationen beschäftigt sich mit der Klassifizierung der zeitlichen Beziehungen zwischen annotierten Ereignissen, mit dem übergeordneten Ziel, logisch konsistente Event-Timelines aus Dokumenten zu erstellen. Dementsprechend müssen vorhergesagte Relationen logische Regeln wie Inversion und Transitivität befolgen. Wie man Sprachmodellen diese logischen Regeln effektiv beibringt, bleibt nach wie vor eine Herausforderung. Während ILP-basierte Ansätze in der Vergangenheit eine beliebte Methode waren, verzichten neuere Ansätze auf post-hoc Korrekturen und zielen stattdessen darauf ab, Sprachmodellen diese Regeln als Soft Constraints mittels differenzierbarer Soft-Logik beizubringen. Daher liegt der Kernfokus dieser Arbeit darauf, die genauen Auswirkungen der Integration solcher Regeln mittels differenzierbarer Soft-Logik während des Trainingsprozesses besser zu verstehen. Um dieser Frage auf den Grund zugehen, fine-tunen wir ein BERT-Base-Modell mithilfe eines Primal-Dual-Kriteriums auf dem TimeBank-Dense-Korpus (TB-Dense). Wir zeigen, dass die Verwendung temporaler Constraints auf diese Weise als effektiver Regulierer fungiert, der Overfitting reduziert und die Varianz über verschiedene Random Seeds hinweg deutlich vermindert. Gleichzeitig kann so auch die Gesamtleistung bei der Klassifikation temporaler Relationen verbessert werden. Anhand einer umfassenden Constraint Ablationsstudie konnten wir zudem komplexe Wechselwirkungen zwischen den verschiedenen Relationsstypen aufdecken. Darüber hinaus stellte sich heraus, dass Sprachmodelle zwar transitive Constraints allein aus den Trainingsdaten heraus lernen können, jedoch explizite logische Hinweise benötigen, um inverse Constraints erfolgreich zu erlernen. Schlussendlich konnten wir noch aufzeigen, dass die Verwendung temporaler Constraints die Dateneffizienz drastisch verbessert: Unser auf Constraints trainiertes Modell, das nur auf 50% der verfügbaren Daten trainiert wurde, performt genau so gut wie ein Modell, das ohne Constraints auf dem vollständigen Datensatz trainiert wurde.

Abstract

The task of temporal relation extraction (TRE) involves classifying temporal relations between annotated event pairs, with the overall goal of creating logically consistent event timelines from documents. Consequently, predicted relations must satisfy constraints such as inversion and transitivity. How to effectively teach language models these logical rules remains an active challenge. While ILP-based methods have been prominent in the past, recent approaches step away from post-hoc corrections, aiming instead to teach models to follow these rules as soft constraints via differentiable soft logic. Therefore, the primary objective of this thesis is to uncover the precise effects of integrating temporal rules via differentiable soft logic during model training. We fine-tune a BERT-base model using a primal-dual objective on the TimeBank-Dense (TB-Dense) corpus. We demonstrate that incorporating temporal constraints this way acts as an effective regularizer, significantly reducing overfitting and variance across random seeds while improving overall task performance. A comprehensive constraint ablation study reveals complex interaction effects between relation types. Furthermore, we show that while language models can inherently learn transitive constraints from training data alone, they require explicit logical guidance to successfully learn inverse constraints. Finally, we demonstrate that integrating constraints drastically improves data efficiency, allowing a constrained model trained on only 50% of the available data to match the performance of an unconstrained model trained on the full dataset.

1 Introduction

“We apprehend time only when we have marked motion, marking it by ‘before’ and ‘after’; and it is only when we have perceived ‘before’ and ‘after’ in motion that we say that time has elapsed.”

— Aristotle, **Physics**, Book IV, ch. 11

This notion that time perception depends on observing motion as ‘before’ and ‘after’ is deeply rooted in human language. It is this kind of event ordering that we encounter continuously in our thoughts, conversations, and writings. While this cognitive task was long performed solely by humans, the advance of artificial intelligence—and more recently, Large Language Models (LLMs)—has increasingly transferred this responsibility to machines.

Within Natural Language Processing (NLP), temporal information extraction encompasses any task focused on retrieving time-related data from text (Su, Howard, & Bethard, 2025). This field is traditionally divided into three sub-tasks: time expression identification, time expression normalization, and temporal relation extraction (TRE) (Pustejovsky et al., 2003). Time expression identification focuses on annotating specific tokens that serve as event markers, while normalization grounds concrete time expressions, such as dates, to standardized formats. Finally, TRE is the task of classifying the chronological relationship between two identified events, e_1 and e_2 , within an input sequence. For example, given the text, “*The First World War started (e_1) after the Sarajevo assassination (e_2),*” a TRE classifier is tasked with determining that e_2 occurred BEFORE e_1 . In this thesis, we focus exclusively on the task of TRE.

The overall goal of TRE is not merely to predict relations between singular event pairs, but to produce a complete timeline including all events from a document. This can be done globally (predicting the whole timeline at once), or by aggregating singular predictions. Looking at the events as nodes and their relations as edges of a temporal graph, they are subject to certain logical constraints: inversion¹ and transitivity. Considering our two events from before, it follows from the fact that the assassination in Sarajevo (e_2) happened BEFORE WWI (e_1) that WWI (e_1) began AFTER Sarajevo (e_2). This is referred to as an inverse constraint. For our next example, let us consider the three events of WWI (e_1), the Great Depression of 1929 (e_2), and WWII (e_3). Imagine a system not knowing the dates of those events. If it is given that e_1 happened BEFORE e_2 and that e_2 happened BEFORE

¹We use “inverse constraints” as an umbrella term to cover both asymmetric inverse pairs (e.g., *Before/After*) and symmetric relations (e.g., *Simultaneous*), where symmetry is defined as a self-inverse constraint.

e_3 , it can conclude from those facts alone that e_1 happened BEFORE e_3 . This is known as transitivity.

Research from recent years suggests that many TRE approaches struggle with these constraints even today (Alfattni, Peek, & Nenadic, 2020; Kougia, Sedova, Stephan, Zaporjets, & Roth, 2024; Leeuwenberg & Moens, 2019; Lim, Jeong, & Choi, 2019; Olex & McInnes, 2021). Treating TRE strictly as a local pairwise classification task frequently produces logically inconsistent temporal graphs—a vulnerability that persists even in the latest generation of LLMs. To address this, various methods have been proposed, most notably enforcing constraints post-hoc via ILP, or using learning-based strategies that incorporate temporal rules into the training process as soft constraints. While ILP has been relied upon for the past two decades to guarantee logical consistency, the underlying effects of embedding rules directly into model training remain critically underexplored. Most studies that employ soft-logic approaches focus exclusively on overall task performance and fail to report actual constraint satisfaction rates. Consequently, the internal dynamics of how models learn and balance these temporal constraints require much deeper investigation.

Therefore, the primary objective of this thesis is to uncover the precise effects of integrating temporal rules via differentiable soft logic during model training. To systematically investigate these dynamics, we focus on the following research questions:

- **RQ1:** How does the integration of symmetry and transitivity constraints via differentiable soft logic during training affect the overall task performance and training dynamics of fine-tuned TRE models?
- **RQ2:** Are the effects of temporal constraints uniformly distributed across all temporal relation types, or do specific constraints exhibit distinct behavioral patterns and interaction effects?
- **RQ3:** How does the inclusion of temporal constraints affect the logical consistency of generated temporal graphs compared to baseline models, and how do inverse and transitive constraints differ in terms of constraint satisfaction rates?
- **RQ4:** Can the explicit integration of logical constraints compensate for a lack of training data in TRE tasks?

To answer these research questions, we fine-tune a BERT-base model using a primal-dual objective on the TimeBank-Dense (TB-Dense) corpus (Cassidy, McDowell, Chambers, & Bethard, 2014).² The primary advantage of this approach lies in its ability to dynamically learn individual weights for each modeled constraint. To establish a baseline for comparison, we train unconstrained models on the same dataset. Furthermore, to gain a deeper understanding of relation-specific constraint dynamics, we conduct two targeted ablation

²The code of our implementation is available on GitHub at <https://github.com/Velkymoss/TempQChainDomiknows.git>.

studies: one set of models is trained with constraints enabled for only a single relation type, while another set utilizes a leave-one-out approach where constraints for a specific relation are disabled. Finally, to address whether constraints can compensate for a lack of training data, we evaluate both a constrained model and a baseline model trained on only 50% of the available data.

In this thesis, we make the following main contributions:

- We demonstrate that incorporating temporal constraints via a primal-dual objective acts as an effective regularizer, significantly reducing overfitting and variance across random seeds while improving overall task performance.
- We provide a comprehensive constraint ablation study, revealing complex interaction effects between relation types, particularly highlighting that directional relations benefit most from global constraints.
- We establish that while language models can inherently learn transitive constraints from training data alone, they require explicit logical guidance to successfully learn inverse constraints.
- We prove that integrating soft-logic constraints drastically improves data efficiency, allowing a constrained model trained on only 50% of the available data to match the performance of an unconstrained model trained on the full dataset.

The remainder of this thesis is structured as follows: Chapter 2 provides a historical overview of TRE and temporal reasoning, contextualizing our work within current research trends. Chapter 3 details our methodological approach, while Chapter 4 describes the technical implementation of our system. Chapter 5 outlines the experimental setup, including the dataset, data splits, hyperparameters, and evaluation metrics. Chapter 6 presents the results of our experiments while chapter 7 provides their analysis. Finally, the concluding chapter summarizes our key findings and suggests directions for future research.

2 The Evolution of Temporal Relation Extraction and Temporal Reasoning

2.1 Early Linguistic and Rule-Based Foundations

Temporal relation extraction is the task of classifying the chronological relationship between two identified events, e_1 and e_2 , within a text. The beginnings of the research field of temporal relation extraction can be traced back to the 1950s with a focus on general-domain texts. Subsequent work until the early 2000s focused on temporal discourse linguistics, temporal information representation, temporal named entity recognition, temporal event ordering, and the development of standardized annotation guidelines. One of the most important domains of the field—the clinical domain—was established in the 1980s, but struggled to keep up with general research trends due to lack of high-quality annotated clinical data (Olex & McInnes, 2021; L. Zhou & Hripcsak, 2007).

Regarding the formal representation of temporal relations, the most influential conceptual work for the field was presented by Allen (1983), whose formal interval algebra—consisting of 13 mutually exclusive relations—provided the foundational logical framework for representing relationships between temporal entities in computational linguistics (see Table 2.1).

Table 2.1: Allen’s 13 Interval Relations. Adapted from Allen (1983)

Relation	Symbol	Inverse	Illustration (Intervals A and B)
Before	$<$	After ($>$)	AAA BBB
Meets	m	Met-by (mi)	AAABBB
Overlaps	o	Overlapped-by (oi)	AAA BBB
Starts	s	Started-by (si)	AAA BBBBB
During	d	Contains (di)	AAA BBBBBBB
Finishes	f	Finished-by (fi)	AAAA BBBBBBB
Equals	$=$	Equals ($=$)	AAA BBB

Many subsequent works employed linguistic heuristics and manually defined constraints

based on Allen’s conceptualization of temporal events (Lim et al., 2019). While those systems tend to perform well in their specialized domains and are less data-hungry than more modern approaches, they usually do not generalize well and their success depends on the developers’ domain expertise and knowledge of linguistics (Gupta, 2015).

2.2 From Annotation Standardization to Supervised Learning

In the early 2000s, the standardization of annotation languages led to a greater availability of gold-standard annotated corpora, which in turn resulted in efforts to solve the task using supervised machine learning (Lim et al., 2019). The first impactful guideline was TIDES (Ferro, Mani, Sundheim, & Wilson, 2001), which focused on the annotation of temporal expressions (expressions that can be mapped onto a timeline). However, because of its exclusive focus on temporal expressions, TIDES omitted expressions lacking sufficient information for timeline mapping, and it did not include tags for events and their relationships. These limitations were addressed with the creation of TimeML (Olex & McInnes, 2021).

Building on the TIMEX2 tag from TIDES, Pustejovsky et al. (2003) introduced TimeML, which annotates time expressions, events, and the relationships between them. The distinction between events and time expressions is crucial: Because the relations between time expressions can be inferred via date normalization, they provide a ground truth against which event predictions can be evaluated for logical consistency.

Alongside their foundational annotation framework, Pustejovsky et al. (2003) launched TimeBank, a corpus consisting of 300 English news articles annotated using the TimeML framework which adapted Allen’s 13 defined temporal relations. Subsequent developments, such as the TempEval series, provided a simplification of Allen’s categorization system and annotation of multilingual data (UzZaman et al., 2013; Verhagen et al., 2007; Verhagen, Saurí, Caselli, & Pustejovsky, 2010).

Driven by standardization efforts and the growing availability of high-quality annotated corpora, the use of machine learning for TRE steadily increased (Lim et al., 2019). Early approaches formulated the task primarily as a local pairwise classification problem, using manually engineered lexical, syntactic, and semantic features (Bethard, 2007; Chambers, Wang, & Jurafsky, 2007; Mani, Verhagen, Wellner, Lee, & Pustejovsky, 2006).

Despite these advances, early machine learning models had notable drawbacks. Although they generally outperformed rule-based systems in relation extraction, they relied heavily on labor-intensive manual feature engineering (Alfattni et al., 2020; Lim et al., 2019; Olex & McInnes, 2021). Furthermore, these approaches treated the problem as an independent local pairwise classification task. This often resulted in the construction of inconsistent temporal graphs that violated logical constraints, such as transitivity (Alfattni et al.,

2020; Leeuwenberg & Moens, 2019; Lim et al., 2019; Olex & McInnes, 2021).

2.3 From Local Classifications to Temporal Reasoning

First attempts at solving inconsistent temporal graphs were already undertaken as early as 2006. In their review paper on temporal reasoning for temporal relation extraction Leeuwenberg and Moens (2019) identified several approaches for addressing the inconsistency problem:

- Best-First (BF) or Natural Reading Order (NRO) Greedy Inference
- Post-Hoc Conflict Resolution
- Sieve-Level Inference
- Stacked Inference
- Integer Linear Programming (ILP)
- Markov Logic Networks (MLN)
- Structured Perceptron with ILP.

In 2006, [Bramsen, Deshpande, Lee, and Barzilay](#) introduced two approaches which applied temporal reasoning during inference. Both methods use an algorithm for determining an ordering for predicted relations coming from a local classifier in a first step: BF orders predictions by confidence (from highest to lowest). NRO processes relations according to their natural reading order. In a second step, the relations are added one-by-one to the temporal graph, followed by a transitive closure algorithm after each added prediction.

For post-hoc conflict resolution, a group of works focused on removing conflicting edges from the graph based on model confidence ([Cheng, Anick, Hong, & Xue, 2013](#); [Meng, Rumshisky, & Romanov, 2017](#); [Sun, 2014](#); [Tatu & Srikanth, 2008](#); [Verhagen & Pustejovsky, 2008](#); see [Leeuwenberg & Moens, 2019](#) for a comprehensive comparison).

An approach similar to BF and NRO Greedy Inference was developed by [Chambers, Cassidy, McDowell, and Bethard \(2014\)](#). In Sieve-Level Inference, a sieve is defined as a model or rule that generates predictions based on both the input text and the output of the preceding sieve. During inference, multiple sieves are applied sequentially. After each sieve, a temporal closure algorithm is executed. Notably, relations are added in groups rather than one by one.

The methods discussed so far rely on explicit knowledge for temporal reasoning during inference. Stacked Inference ([Laokulrat, Miwa, & Tsuruoka, 2014, 2016](#); [Meng & Rumshisky, 2018](#)), on the other hand, learns label associations directly from the data, meaning logical consistency is not deterministically enforced. In the first step, a local classifier generates pairwise predictions. In the next step, a second model takes the original input text and these initial predictions along with their associated probabilities as input

to produce the final output. The advantage of this approach lies in the corrective power of the second model, which can resolve inconsistencies introduced by the local classifier (Leeuwenberg & Moens, 2019). Ultimately, this method differs from the aforementioned approaches in that it relies on the model to learn constraints from the data itself rather than applying a brute-force approach during inference.

Another popular approach for enforcing logical consistency was the use of ILP, first introduced for temporal relation extraction by Bramsen, Deshpande, Lee, and Barzilay (2006b). This method uses the predictions of a local classifier as input; the solver then searches for the combination of labels that maximizes overall confidence without violating any constraints. However, early applications of ILP only used a small subset of Allen’s original 13 relations, as incorporating the full set creates high computational complexity for an ILP solver (Leeuwenberg & Moens, 2019). Denis and Muller (2011) addressed this limitation by reformulating predictions using point algebra rather than Allen’s interval algebra, significantly reducing the required number of constraints. After the ILP optimization step, these point-wise decisions are converted back into interval-based relations. Overall, this approach is guaranteed to find the global maximum within the space of satisfied constraints, though its efficiency heavily depends on the total number of enforced constraints.

As another non-deterministic method, Markov Logic Networks (MLNs) assign learned weights to each constraint, framing them as soft constraints where violations remain possible but less likely (Yoshikawa, Riedel, Asahara, & Matsumoto, 2009). Unfortunately, MLNs suffered limited popularity due to severe scalability issues, as the approach does not cut off large sections of the search space in advance to optimize efficiency (Leeuwenberg & Moens, 2019).

Finally, the Structured Perceptron with ILP integrates global constraint optimization directly into the model’s training loop. First introduced by Abend, Cohen, and Steedman (2015), this approach formulates TRE as a structured learning problem. During inference, the ILP solver enforces hard logical constraints on top of the perceptron’s scored predictions to decode the optimal valid temporal graph. The perceptron then compares this constrained output against the ground truth to update its weights. Compared to other approaches, this method does not ensure constraints deterministically. It learns label associations guided by the ILP-solutions rather than learning the rules directly.

Overall, this era produced a clear divide between inference-based and learning-based methods for resolving inconsistent temporal graphs. While inference-based approaches excel at enforcing hard logical constraints, they suffer from high latency and poor scalability. On the other hand, learning-based methods provide faster inference but fail to guarantee strict consistency, because they capture statistical label associations rather than underlying logical rules. Frameworks like MLNs that attempted to learn these rules as soft constraints faced severe scalability issues. Ultimately, the field lacked an architecture capable of combining the strict logical guarantees of inference-based solvers with the efficiency and

adaptability of learning-based approaches.

2.4 The Neural Turn

Around 2017, the field transitioned away from feature-engineering-based machine learning methods toward the use of word embeddings combined with deep learning (Alfattni et al., 2020; Olex & McInnes, 2021). Dligach, Miller, Lin, Bethard, and Savova (2017) demonstrated that convolutional neural networks (LeCun & Bengio, 1998) and LSTMs (Hochreiter & Schmidhuber, 1997) operating purely on tokens as input could outperform state-of-the-art hand-engineered models. Similarly, Tourille, Ferret, Neveol, and Tannier (2017) were among the first to show that a bidirectional LSTM architecture performed better than traditional machine learning methods in the clinical domain. In terms of temporal reasoning, however, the field saw limited progress during that time. Models were trained as pairwise local classifiers for temporal relations without much focus on logical constraints (Leeuwenberg & Moens, 2019). Overall, this short era marked the entry of deep learning as the dominant paradigm in the field, eliminating the need for manual feature engineering.

2.5 The Rise of the Transformers

The introduction of the transformer architecture (Vaswani et al., 2017) and powerful encoder models like BERT (Devlin, Chang, Lee, & Toutanova, 2019) marked another major paradigm shift in TRE. Early on, Han, Hsu, et al. (2019) demonstrated that BERT embeddings significantly outperformed traditional GloVe embeddings (Pennington, Socher, & Manning, 2014), highlighting the critical advantage of contextualized representations over static ones. In the subsequent years, advancements in transformer-based extraction diverged into two areas: integrating external knowledge and enhancing model robustness.

To integrate external knowledge, research has focused on incorporating commonsense knowledge, syntactic and semantic knowledge, knowledge of label distributions, and temporal-specific rules. Concurrently, model robustness has been largely advanced through multi-task learning, continued pre-training, adversarial training, and self-training (Su et al., 2025). Because this study investigates the impact of integrating explicit temporal reasoning into the training process, the remainder of this section specifically examines the subset of literature focused on incorporating temporal rules and logical constraints into these models.

Han, Hsu, et al. (2019); Han, Ning, and Peng (2019) enforce constraints using an ILP solver during training and inference. After generating contextualized representations of the events (BERT+LSTM), they use ILP to resolve conflicts before passing the solver's solution to a structural SVM. During the first stage, the model is trained as a local classifier, while the SVM is trained as a global model during the second stage. While the feedback from

the solver during training helps the model learn constraints, the dependence on the solver during inference remains a major bottleneck of this approach.

Another group of works focuses on applying logical constraints through custom heuristics. [Liu, Xu, Chen, and Zhang \(2021\)](#) rely on a Certain-First Graph Completion, which leads to good constraint satisfaction rates but underperforms on local temporal datasets. [J. Zhou, Dong, Tu, Wang, and Dou \(2022\)](#) resolve logical inconsistencies with a cycle-reversal heuristic. Unfortunately, this method yields only minimal performance gains compared to fully trained models. [L. Wang, Li, and Xu \(2022\)](#) rely on Document-Creation-Time-Guided global inference to establish consistent timelines. Of these rule-based approaches, only [Liu et al. \(2021\)](#) provide an analysis of constraint satisfaction rates, showing that their method performs only slightly better than a BERT-based classifier coupled with an ILP solver.

[Hwang et al. \(2022\)](#) and [Tan, Pergola, and He \(2021\)](#) encode constraints directly into the embeddings, building on the assumption that Euclidean space cannot capture asymmetric relations. [Hwang et al. \(2022\)](#) use probabilistic box embeddings and calculate their conditional volume overlap. Their approach guarantees no inverse violations by design and keeps transitivity violations at a low rate. However, compared to approaches that enforce constraints via differentiable soft-logic, box embeddings do not possess the flexibility to prioritize task performance over strict constraints, leading to worse task performance relative to more flexible methods on certain benchmarks. [Tan et al. \(2021\)](#) rely on a similar approach utilizing hyperbolic embeddings: they embed events and their relations in hyperbolic spaces, which excel at modeling hierarchical structures. Unfortunately, the method was evaluated solely on task performance, making an evaluation of its constraint satisfaction rate infeasible.

Turning to differentiable soft-logic, [H. Wang, Chen, Zhang, and Roth \(2020\)](#) and [Y. Zhou et al. \(2021\)](#) teach the model temporal rules via soft constraints added to the model's loss. While [H. Wang et al. \(2020\)](#) rely on the product t-norm, [Y. Zhou et al. \(2021\)](#) utilize the Łukasiewicz t-norm to transform temporal rules into differentiable regularization terms. Additionally, both works report the use of post-processing approaches to ensure logical consistency: [H. Wang et al. \(2020\)](#) include an optional ILP post-processing step but note that it adds minimal performance gains, whereas [Y. Zhou et al. \(2021\)](#) use a timegraph algorithm to ensure transitivity during post-processing. Ultimately, within the broader literature, these two works represent the approaches closest to our own method, differing primarily in one important aspect.

Compared to [H. Wang et al. \(2020\)](#) and [Y. Zhou et al. \(2021\)](#), our approach (see Section 3) utilizes a primal-dual objective. While their constraint learning relies on a singular, fixed parameter to indicate the importance of the global constraint loss during training, our objective uses dual variables. This means that each modeled constraint possesses its own Lagrange multiplier, a learned parameter that determines the importance of each individ-

ual rule during training. In other words, our approach is much more flexible, granting the model the ability to learn the importance of each rule individually. Due to this fundamental difference in our optimization process, we expect better performance on asymmetric constraints compared to the approach of [H. Wang et al. \(2020\)](#), as observed in the constraint satisfaction analysis provided by [Hwang et al. \(2022\)](#).

However, while these highly optimized encoder frameworks represent the peak of supervised constraint learning during the Transformers era, the recent generation of Large Language Models introduces a completely different caliber of models for the task of TRE. Whether these generative models inherently possess the capacity for temporal reasoning is explored in the next section.

2.6 Can Large Language Models Perform Temporal Reasoning?

Following the release of ChatGPT in late 2022, prompting large language models (LLMs) quickly became the standard approach for many NLP tasks. However, TRE has proven to be a notable exception to this paradigm shift. Early zero-shot evaluations revealed a substantial performance gap between LLMs and state-of-the-art supervised models ([Yuan, Xie, & Ananiadou, 2023](#)). This underperformance also persists for in-context learning ([Rocabruna, Rizzoli, & Riccardi, 2024](#)) and extends to specialized fields such as the clinical domain ([Kougia et al., 2024](#)). Additionally, [Kougia et al. \(2024\)](#) demonstrated that LLMs struggle with uniqueness and transitivity constraints. While enforcing these constraints post-hoc via ILP yields logically consistent predictions, it can ultimately degrade overall task performance.

Subsequent studies suggest that LLMs possess latent temporal reasoning abilities when appropriately guided. For instance, [Xiong, Payani, Kompella, and Fekri \(2024\)](#) demonstrated that LLMs perform significantly better on temporal reasoning tasks when trained to first transform the input sequence into a temporal graph. Conversely, [Fatemi et al. \(2025\)](#) found that when asked to perform temporal reasoning directly, LLMs rely heavily on parametric memory and struggle with chronological sorting and factual ordering. These limitations strongly reinforce the conclusion that LLMs cannot excel at temporal reasoning without fine-tuning or structural interventions like ILP.

Recent works validate this assumption. [Eirew, Bar, and Dagan \(2025\)](#) proposed a prompting approach that generates a comprehensive temporal graph in a single step, followed by ILP post-processing. They achieved competitive performance compared to state-of-the-art methods on dense benchmarks. Crucially, their constraint analysis revealed that the ILP step significantly improves transitive consistency without degrading performance, whereas the approach without ILP produces inconsistent graphs. Similarly, [Homburger and](#)

[Bar \(2025\)](#) fine-tuned an LLM for global temporal relation extraction, outperforming state-of-the-art baselines across multiple benchmarks. Their ablation studies indicate that this global model produces significantly fewer constraint violations than a pairwise baseline. However, because they only report absolute numbers without comparing constraint violations against other baselines, the intrinsic reasoning capabilities of their approach remain difficult to fully assess.

Ultimately, LLMs exhibit significant potential for temporal relation extraction when fine-tuned or augmented with ILP. Without these interventions, however, their temporal reasoning capabilities pale in comparison to smaller, fine-tuned encoder models. This naturally raises a question of computational efficiency: there is little incentive to deploy a resource-heavy LLM if a smaller model performs equally well or better. Nevertheless, in scenarios where no training data is available, the work of [Eirew et al. \(2025\)](#) demonstrates that LLMs can serve as a highly viable alternative to specialized supervised models.

3 Methodology

3.1 Constraints

One of the primary goals of this thesis is to ensure that the model learns to respect the underlying logical constraints governing temporal event relations. Before diving into the training process, we must first formalize the definition of each temporal relation, along with its corresponding inverse and transitivity rules.

When talking about definitions of temporal relations, most frameworks relate back to Allen’s interval algebra (Allen, 1983). As stated by Ning, Feng, Wu, and Roth (2018), most annotation schemas use a reduced form of Allen’s framework, since a reduced set of labels usually improves performance. Despite widespread adoption, formal definitions of reduced relations are often overlooked. Ning et al. (2018) introduced a revised scheme in which low-frequency relations are absorbed into their temporally closest neighbors (see Table 3.1). They note that while such choices have minimal impact on single event pair classification, they directly dictate the composition of transitive rule sets ($\text{Trans}(r_1, r_2)$). Because this framework offers the most complete and rigorous rationale for the label set of our benchmark (TB-Dense), we adopt this schema as the foundational axiom undermining our transitive constraints.

In addition to transitivity, inverse relations represent essential constraints that the model must learn. As discussed in Section 3.4, these inverse relations also directly influence the construction of transitive reasoning chains. For our experiments, we incorporate all potential inverse relations—including symmetric ones—with the exception of the VAGUE relation. The rationale for excluding VAGUE will be explained alongside the transitive rule set, where all rules involving VAGUE were likewise cut.

Regarding transitive constraints, we restricted our rule set exclusively to those yielding a fully deterministic conclusion. Consequently, rules whose premises do not reduce the set of valid candidate relations down to a single outcome are excluded during model training (see Table 3.3). This decision stems from the observation that non-deterministic compositions send conflicting optimization signals, causing training instability. For instance, given the premise pair BEFORE–INCLUDES, two distinct instances with identical premises might have different ground-truth labels (e.g., BEFORE vs. INCLUDES). While the constraint-based loss does not penalize predicting either valid candidate, the primary task loss penalizes any prediction that diverges from the specific instance’s target. This mismatch creates confusing

feedback for identical relational inputs. As a consequence, incorporating non-deterministic rules severely degraded performance during early model development; restricting the training set to strictly deterministic transitive rules yielded a substantial performance improvement.

As already mentioned, we omitted all rules containing the *VAGUE* relation. This choice was driven by a severe performance drop when introducing constraints with *VAGUE*. Specifically, the model collapsed into predicting only the majority class, completely ignoring all other relations. We hypothesize two main reasons for this behavior. First, *VAGUE* constitutes the vast majority of the dataset (nearly half of all annotated relations). Emphasizing this relation in any form appears to be sufficient enough to bias the model entirely toward the majority class. Second, this behavior may stem from the annotation process itself. *VAGUE* is typically assigned when inter-annotator agreement for an event pair is low, meaning it lacks a clear semantic definition compared to the other categories. This ambiguity likely sends conflicting signals to the model during training. Ultimately, a combination of class imbalance and noisy training signals most likely causes the model to over-predict the majority class whenever *VAGUE* is incorporated into constraint rules.

Table 3.1: Temporal relation definitions between intervals A and B . Adapted from Ning et al. (2018).

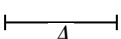
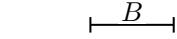
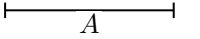
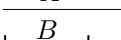
Label	Interval Alignment (A relative to B)	Relation Description
before		A ends before or as B starts
		
		
		
includes		A starts before B , ends together
		
		
		
is included		A starts with B , ends before
		
		
		
simultaneous		A and B are identical
after		A overlaps end of B
		
		
		
vague	?	Annotated when no unique well-defined relation can be assigned (omitted from logical constraints)

Table 3.2: Inverse mapping for temporal relations $R(A, B) \iff R^{-1}(B, A)$.

Relation (R)	Inverse Relation (R^{-1})
before	after
after	before
includes	is included
is included	includes
simultaneous	simultaneous (<i>symmetric</i>)
vague	vague (<i>symmetric</i>)

Table 3.3: Transitive composition rules for temporal relations $R_1(A, B) \circ R_2(B, C) \implies R_3(A, C)$.

Relation 1 (R_1)	Relation 2 (R_2)	Possible Resulting Relations (R_3)
<i>Fully Deterministic Rules (Single Outcome)</i>		
before	before	{before}
after	after	{after}
includes	includes	{includes}
is included	is included	{is included}
simultaneous	simultaneous	{simultaneous}
before	simultaneous	{before}
after	simultaneous	{after}
includes	simultaneous	{includes}
is included	simultaneous	{is included}
simultaneous	before	{before}
simultaneous	after	{after}
simultaneous	includes	{includes}
simultaneous	is included	{is included}
<i>Partially Deterministic Rules (Deterministic Subsets)</i>		
before	includes	{before, includes}
before	is included	{before, is included}
after	includes	{after, includes}
after	is included	{after, is included}
includes	before	{before, includes}
includes	after	{after, includes}
is included	before	{before, is included}
is included	after	{after, is included}

3.2 Objective and Training Algorithm

Each target question x in a training batch could be logically related to other questions by a constraint label (e.g., INVERSE or TRANSITIVE). During training, we evaluate model predictions across the entire batch to detect possible constraint violations. For each constraint-targeted question, a penalty is added to the loss when a prescribed rule is violated. This section details the derivation of the training objective and the resulting optimization algorithm.

First, we formulate the global loss $L(w)$ as a weighted combination of the primary task loss $L_{\text{task}}(w)$ and a constraint loss $L_{\text{constraint}}(w)$:

$$L(w) = L_{\text{task}}(w) + \gamma \cdot L_{\text{constraint}}(w) \quad (3.1)$$

where $\gamma > 0$ is a parameter balancing the overall impact of the constraint loss. To account for varying difficulty and importance across individual constraints, we introduce a learned Lagrange multiplier $\lambda_k \geq 0$ for each constraint $k \in \{1, \dots, K\}$. The joint constrained optimization objective is given by the following Lagrangian formulation:

$$\mathcal{L}(w, \boldsymbol{\lambda}) = L_{\text{task}}(w) + \gamma \sum_{k=1}^K \lambda_k h_k(w) \quad (3.2)$$

where $h_k(w) \geq 0$ measures the degree of violation for the k -th constraint rule over the current batch, and $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_K]^\top$ represents the vector of constraint weights. For our final objective we convert the combined loss into a primal-dual objective:

$$\max_{\boldsymbol{\lambda} \geq 0} \min_w \mathcal{L}(w) + \sum_{k=1}^K \lambda_k [h_k(w)]^+ \quad (3.3)$$

This primal-dual formulation—which integrates hard constraints into training via differentiable soft logic—was first introduced by [Nandwani, Pathak, Singla, and Singla \(2019\)](#). In addition to reducing constraint violations and enhancing task performance, it guarantees convergence to a local min-max point.

Before we can put everything together for the final training algorithm, we first need to clarify the calculation of $h_k(w)$. To evaluate the constraint penalty $h_k(w)$, logical rules are mapped to continuous truth values in $[0, 1]$ using the Product T-norm. A conjunction ($A \wedge B$) is given by $a \cdot b$ and an Implication ($A \implies B$) by $\min(1, \frac{b}{a})$, where A and B represent labels of pairs of events with probabilities a and b each. For a transitivity rule over a relation chain with probabilities a, b , and c , the constraint violation $T(a, b, c)$ is modeled as an implication

from the premise $a \wedge b$ to the conclusion c :

$$T(a, b, c) = \max \left(0, \frac{a \cdot b - c}{a \cdot b} \right) \quad (3.4)$$

Similarly, for an inverse rule $R_1(A, B) \implies R_2(B, A)$ with probabilities a and b , the inverse constraint violation $I(a, b)$ is modeled as an implication from the premise a to the conclusion b :

$$I(a, b) = \max \left(0, \frac{a - b}{a} \right) \quad (3.5)$$

Finally, for the total penalty $h_k(w)$ for a given constraint type k , we sum up and normalize all its related individual hinge losses $H(K_{k,i})$ from the current batch:

$$h_k(w) = \frac{1}{N_k} \sum_{i=1}^{N_k} H(K_{k,i}) \quad (3.6)$$

Now after clarification of our training objective we can finally consider the full picture of the training algorithm. We used an adapted version of the original algorithm developed by [Nandwani et al. \(2019\)](#) based on the implementation of [Rajaby Faghihi, Guo, Uszok, Nafar, and Kordjamshidi \(2021a\)](#). Algorithm 1 displays the training algorithm used for optimizing our primal-dual objective:

Algorithm 1 Training Algorithm. Hyperparameters: $warmup, d, \alpha_\lambda, \alpha_w$

```

1: Initialize:  $w$  randomly;  $\lambda_k = 0, \forall k = 1, \dots, K$ 
2: for  $i = 1$  to  $warmup$  iterations do
3:   Update  $w$ : Take an ADAMW step wrt  $w$  on  $\mathcal{L}(w; \lambda)$  on a batch.
4: end for
5: Initialize:  $l = 1, t = 1$ 
6: while not converged do
7:   Update  $\lambda$ : Take an ADAM step wrt  $\lambda$  on  $\mathcal{L}(w; \lambda)$  on a batch.
8:   Increment  $t = t + 1$ 
9:   for  $i = 1$  to  $l$  steps do
10:    Update  $w$ : Take an ADAMW step wrt  $w$  on  $\mathcal{L}(w; \lambda)$  on a batch.
11:   end for
12:   Update  $l = l + d$ 
13:   Set learning rates:  $\alpha_\lambda = \alpha_\lambda \sqrt{\frac{t+1}{t+2}}$ 
14: end while

```

The algorithm takes as input the primal weights w , dual multipliers λ , the number of warm-up iterations, the constraint update increase rate d and initial learning rates α_w and α_λ . First, we initialize w randomly and λ_k with zeroes. After that, we update the primal parameters w for the number of $warmup$ iterations. Then until the algorithm converges, for every update of λ , we update w for l iterations, where l is increased by the constant d before each update of λ . At the end of each cycle, we update α_λ by multiplying it with a

learning rate scaling factor that asymptotically approaches one, resulting in progressively smaller changes to the learning rate. For optimizers, we use ADAMW for the primary model parameters w , as it represents the standard choice for transformer-based architectures. For the dual multipliers λ , we employ the ADAM optimizer, following the implementation guidelines of the DomiKnowS framework (Rajaby Faghihi et al., 2021a).

3.3 Architecture

In this section, we present the Transformer-based architecture used for our experiments. Architectural choices were heavily influenced by the work of Hu, Huang, and Feng (2024). The model leverages a pre-trained encoder (BERT) to capture contextual representations of the event-pair input sequence via specialized marker embeddings. Figure 3.1 illustrates a high-level overview of the architecture.

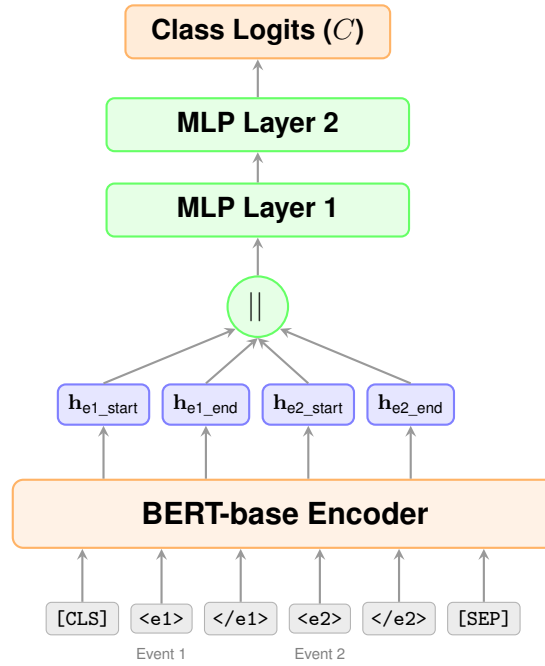


Figure 3.1: Entity-Aware BERT Architecture. Marker embeddings are extracted from output hidden states, concatenated, and passed into a 2-layer MLP classifier.

To make the model aware of the positions of the events within an input sequence, we introduce special entity marker tokens ($\langle e1 \rangle$, $\langle /e1 \rangle$, $\langle e2 \rangle$, and $\langle /e2 \rangle$) around the target events and add them to the model’s vocabulary. The tokenized sequence, prepended with $[\text{CLS}]$ and appended with $[\text{SEP}]$, is passed through a pre-trained BERT-base encoder to construct contextual representation vectors.

Rather than relying solely on the final hidden state of the $[\text{CLS}]$ token, we extract the

output vector representations corresponding to the start and end markers of both events:

$$\mathbf{h} = [\mathbf{h}_{e1_start} \parallel \mathbf{h}_{e1_end} \parallel \mathbf{h}_{e2_start} \parallel \mathbf{h}_{e2_end}] \quad (3.7)$$

where $\mathbf{h} \in \mathbb{R}^{4d_h}$ and d_h represents the hidden dimension size of the Transformer layer. These four vectors are concatenated horizontally to capture the joint context of the event pair.

The concatenated vector $\mathbf{h}$ is then fed into a 2-layer Multilayer Perceptron (MLP) classification head:

1. **MLP Layer 1:** Projects the concatenated representations from dimension $4d_h$ down to d_h , followed by a GELU activation function and a dropout layer for regularization:

$$\mathbf{z}_1 = \text{Dropout}(\text{GELU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1)) \quad (3.8)$$

2. **MLP Layer 2:** Reduces the dimensionality from d_h to $d_h/2$ with GELU activation and dropout, followed by a final linear layer mapping features to the target relation classes C :

$$\mathbf{z}_2 = \text{Dropout}(\text{GELU}(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2)) \quad (3.9)$$

$$\mathbf{y} = \mathbf{W}_3 \mathbf{z}_2 + \mathbf{b}_3 \quad (3.10)$$

Finally, the output class logits $\mathbf{y} \in \mathbb{R}^C$ are passed to a Softmax function during inference to obtain class probabilities, or processed directly via Cross-Entropy Loss during training.

3.4 Data Augmentation

Adding temporal transitivity and inverse constraints to the loss function requires the generation of transitive chains in the training data and the annotation of inverse relations. The algorithm which was used for data augmentation was inspired by [Premisri and Kordjamshidi \(2025\)](#) and adapted for the use of temporal relations. The general idea is to look for direct transitive relations and if that fails, we apply inverse rules for the first and/or second candidate questions of the transitive chain to create a valid transitive relation.

The algorithm (see Algorithm 2) receives an array of event pairs $E(e_1, e_2)$, an array of transitive triplets originating from the same document, an inverse rule function r^{-1} , and a transitive rule table T as input. To begin with, we iterate over the event pairs. First, we examine the third element in the transitive triplet array. If a matching third element is found, we annotate a direct transitive relationship and add the chain to the data. If no direct transitive relationship is identified, we search for all events E containing x (excluding C) as candidates for A , and we extract all pairs containing y as candidates for B . We

Algorithm 2 Rule-Constrained Temporal Reasoning Data Augmentation

Input: Event pairs $E(e_1, e_2)$, transitive triplets array, inverse rule function r^{-1} , transitive rule table T

For each event pair $C(x, z) \in E(e_1, e_2)$:

1. **Check direct transitive match:** Inspect the third elements in the transitive triplets array. If a matching third element is found, annotate the direct transitive relationship, store the chain in the data, and proceed to the next pair.
2. **Identify candidate events:** If no direct relationship was found, search for all events in E containing x (excluding C) as candidates for A , and all pairs containing y as candidates for B .
3. **Evaluate inverse combinations:** Evaluate candidate relation pairs A and B under the three remaining orientation combinations:
 - (1) $A(x, y), B^{-1}(y, z)$
 - (2) $A^{-1}(x, y), B^{-1}(y, z)$
 - (3) $A^{-1}(x, y), B(y, z)$

If a valid transitive chain is found using T , store the chain in the data.

4. **Fallback to inverse relation:** Otherwise, if no valid chain was found and an inverse relation $C^{-1}(z, x)$ exists for C , store the inverse reasoning path.

then evaluate all three inverse combinations of the candidate pairs A or B —inverting either one individually or both simultaneously—to establish a valid transitive chain. If a valid configuration is found, we store the chain in the data. Otherwise, if no chain is identified and an inverse relation C^{-1} exists for C , we store the inverse reasoning path.

In terms of reasoning paths, it is important to specify the number of reasoning steps required for each chain type. Direct transitivity and direct inverse relations each require a single reasoning step. In contrast, inverting one premise within the chain (either A or B) to establish transitivity requires two reasoning steps in total. Finally, when both premises must be inverted, three reasoning steps are needed to complete the full transitive chain.

Importantly, only single-step constraints—namely direct transitive chains and inverse relations—are incorporated into the loss function during model training; for longer reasoning chains, only the direct premise-to-conclusion link is added to the training batch. Nevertheless, the complete reasoning path (including any intermediate inverse operations) is preserved in a path variable and stored alongside the simplified chain in the output dataset, enabling future research to leverage the full step-by-step reasoning trajectories.

Beyond generating the augmented data set, these annotated reasoning chains must be properly structured during training. Specifically, every question with a transitive or inverse constraint must share a training batch with its corresponding chain questions, as our loss function evaluates these constraints on a per-batch basis. The data set construction allows us to achieve this by iterating over questions paired with their relation and constraint labels.

The original batch-building process proposed by [Premisri and Kordjamshidi \(2025\)](#) generates transitive relations throughout a document. Although this approach is effective for

short texts, it fails for our dataset. Because our documents are substantially longer and contain many more annotated events, document-wide chain annotations cause related chain questions to be scattered across different batches, preventing the batch-level constraint loss from computing correctly.

To resolve this, we designed a custom batch processing algorithm (see Algorithm 3). This algorithm guaranties that every target question is placed in the same batch as all of its intermediate constraint questions. As a result, certain questions may appear in multiple batches within a single training epoch.

Algorithm 3 Question Batching

Input: array of questions, batch size

Output: batch[]

```

1: Initialize batch  $\leftarrow []$ , candidate  $\leftarrow \{\}$ 
2: for EACH  $q$  IN questions do
3:   if  $q$  NOT IN current batch then
4:     Add question + related candidate questions to candidate (if chain exists)
5:   else
6:     if  $q$  already in batch with reasoning chain then
7:       continue
8:     else
9:       //  $q$  in batch without reasoning chain - add chain questions
10:      Add missing reasoning chain questions to candidate
11:    end if
12:  end if
13:  if  $\text{len}(\text{batch}) + \text{len}(\text{candidate}) > \text{batch size}$  then
14:    return batch
15:  else
16:    Add candidate to batch
17:  end if
18: end for
19: return batch

```

The algorithm accepts the full array annotated training questions and a designated target batch size as inputs, returning a fully formed training batch. To begin with, we initialize an empty batch array along with a set of candidate questions. We then iterate over each question q in the dataset. If q is not yet present in the current batch, we add both q and its associated chain questions (if a temporal constraint exists) to the candidate set. If q has already been inserted into the batch, we check its status: if it is already present alongside its full reasoning chain, no further action is required and the loop continues; if it was previously added without its chain, we append the missing intermediate chain questions to the candidate set. Finally, before adding the current candidate set to the batch, we check whether doing so would cause the total number of questions to exceed the target batch size. If the batch size threshold would be violated, the current iteration terminates and returns the completed batch, ensuring that every target question and its

related intermediate questions are within the same batch.

It should be noted that this version of the algorithm handles batch overflow suboptimally. When the batch size threshold is exceeded, the remaining candidate questions in the iteration are discarded rather than carried over to the next batch, resulting in a minor loss of usable training data. While an optimal implementation would append these overflowing questions to the subsequent batch upon its initialization, this adjustment was omitted here as the issue was identified only after experimental training and evaluation were completed. Given the negligible volume of affected data, re-running the training pipeline was deemed computationally inefficient relative to the expected performance gain.

4 DomiKnowS Implementation

We implemented our methodology (see chapter 3) via the DomiKnowS library ([Rajaby Faghihi et al., 2021a](#)), a framework which specializes in the integration of domain knowledge into deep learning architectures. This chapter informs the reader about the library and the implementation of our approach with the framework. First, we describe the architecture of the api and the interaction of its components. Second, we display our specific implementation of temporal constraints. The final section explains how we test the correct implementation of the constraints.

4.1 The DomiKnowS Api

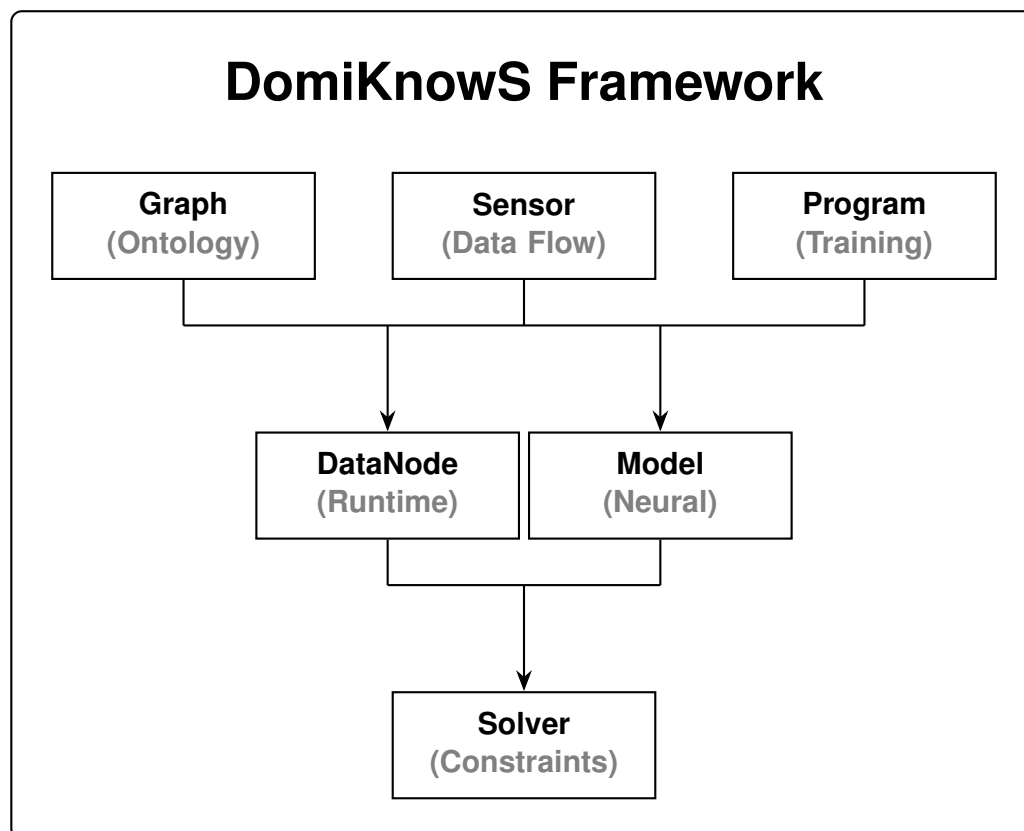


Figure 4.1: DomiKnowS architecture. Adapted from the DomiKnowS GitHub repository ([Rajaby Faghihi et al., 2021b](#)).

One of the core design principles of DomiKnowS is the separation of knowledge declaration from learning algorithms. Users declare rules and constraints in the form of a graph. A DomiKnowS graph consists of nodes representing concepts, edges representing relations between these concepts, and constraints that the represented entities are required to satisfy. As the primary interface for modeling constraints, the graph is connected to the other components of the library and thereby guides the training process. Two additional components, Sensors and Programs, form part of the primary user interface of DomiKnowS.

Data is connected to the concepts in the graph through Sensors, which control the data flow within the application. By combining different Sensors, users can define a pipeline ranging from preprocessing steps to multiple interconnected model layers. Once the graph and its associated Sensors have been defined, these components are brought together inside a Program.

A Program provides a high-level interface for training. It takes the required datasets and training-related hyperparameters as input and coordinates the underlying learning process. The API provides various approaches to constraint-aware learning, ranging from primal-dual methods (see Chapter 3) to ILP-based methods. While this interface considerably simplifies the training process, it offers limited options for customization. We therefore modified the learning-based Program to support early stopping during training and more detailed reporting of training metrics.

The remaining components of the architecture primarily represent lower-level abstractions used internally by DomiKnowS at runtime. DataNodes essentially represent individual data instances and contain the metadata required to connect them to the graph. The library is compatible with most PyTorch components, allowing users to define PyTorch models that can subsequently be wrapped by Sensors and incorporated into a Program. DomiKnowS also employs ILP solvers, primarily Gurobi, for measuring constraint satisfaction, guiding model training, and performing inference. The following section describes how temporal constraints are declared within a DomiKnowS graph.

4.2 Implementing Temporal Constraints with DomiKnowS

The graph provides the primary interface for declaring constraints in DomiKnowS. Figure 4.2 shows a simplified version of the graph used for our primal-dual objective. Before logical rules can be declared, the relevant concepts and their relations must first be defined. In our graph, a `story` represents a document containing annotated event pairs. For each annotated event pair, a `question` represents the corresponding temporal-relation classification problem. We therefore define a `contains` relation between the `story` and `question` concepts. Each question is further associated with an `answer_class`, represented as an `EnumConcept` containing the six temporal relation classes as values.

```

1 with Graph("temporal_QA_rule") as graph:
2     story = Concept(name="story")
3     question = Concept(name="question")
4     (story_contain,) = story.contains(question)
5
6     answer_class = question(
7         name="answer_class",
8         ConceptClass=EnumConcept,
9         values=["after", "before", "includes",
10                "is_included", "simultaneous", "vague"],
11     )
12
13     output_for_loss = question(name="output_for_loss")
14
15     # Inversion: before(A,B) -> after(B,A)
16     inverse = Concept(name="inverse")
17     inverse_question1, inverse_question2 = inverse.has_a(
18         arg1=question, arg2=question
19     )
20
21     ifL(
22         andL(
23             inverse("i"),
24             answer_class.before(
25                 path="i", inverse_question1)
26             ),
27         ),
28         answer_class.after(
29             path="i", inverse_question2)
30     ),
31 )
32
33     # Transitivity: before(A,B) & before(B,C) -> before(A,C)
34     transitive = Concept(name="transitive")
35     tran_quest1, tran_quest2, tran_quest3 = transitive.has_a(
36         arg11=question, arg22=question, arg33=question
37     )
38
39     ifL(
40         andL(
41             transitive("t"),
42             answer_class.before(path="t", tran_quest1)),
43             answer_class.before(path="t", tran_quest2)),
44         ),
45         answer_class.before(path="t", tran_quest3)),
46 )

```

Figure 4.2: DomiKnowS graph definition with inversion rule and transitivity rule.

Inversion and transitivity are defined as compositional concepts. They form `has_a` relationships with their respective arguments: `inverse` is connected to two questions participating in an inverse relationship, whereas `transitive` is connected to three questions forming a transitive chain.

The constraints themselves are expressed using the logical operators `ifL` and `andL`. The former represents a logical implication, while the latter represents a conjunction. The inverse constraint shown in Listing 4.2 can therefore be interpreted as follows: if two questions participate in an inverse relationship and the first question has the relation `before`, then the second question must have the inverse relation `after`. The transitivity constraint follows a similar structure: if three questions form a transitive chain and the first two questions both have the relation `before`, then the third question must also have the relation `before`.

Another key aspect of the syntax involves using the strings `"s"` and `"t"` as instance variables for the `inverse` and `transitive` concepts. These strings can then be passed to the `path` argument of a temporal relation, connecting it to the constraint concept and its related questions. In other words, these variables serve as the links that connect concepts across the knowledge graph.

Overall, the DomiKnowS framework has a steep learning curve, and its abstractions can initially be difficult to grasp. Consequently, it can be challenging to verify whether the modeled constraints are functioning correctly. Furthermore, the official documentation provides limited guidance on testing such graphs. However, upon contacting the developers, they provided a template for designing an integration test. In the following section, we present an adapted integration test for our temporal constraint graph.

4.3 Integration Testing of Constraints with DomiKnowS

The core principle of integration testing constraint graphs within the DomiKnowS framework is to generate deterministic dummy predictions that intentionally violate a specified logical constraint. An Integer Linear Programming (ILP) solver is then tasked with correcting these predictions to comply with the constraint graph. If the constraints are implemented correctly, the solver will successfully adjust the predictions to satisfy the predefined rules.

Algorithm 4 outlines this procedure. The algorithm takes a small synthetic dataset as input, consisting of questions specifically constructed to form a symmetric or transitive chain. Typically, when testing a single constraint, this dataset consists of a single batch. Initially, a dummy model generates deterministic predictions designed to deliberately violate the target constraint. For instance, in a transitive chain, the first two questions are assigned correct predictions to satisfy the premise, while the conclusion is intentionally assigned an incorrect prediction. Because the constraint is modeled as a logical implication, it is only

violated when the premise is true and the conclusion is false. The dummy predictions strictly emulate this condition. Once these intentionally flawed predictions are verified, the ILP solver attempts to infer a solution that satisfies the graph's constraints. If the solver successfully resolves the violation, the integration test passes.

Algorithm 4 Integration Test for Temporal Constraint Graph

Input: dataset

```

1: for EACH batch IN dataset do
2:   for EACH question IN batch do
3:     Get dummy prediction
4:     Check correctness of dummy prediction
5:   end for
6:   Infer ILP results for batch questions
7:   Check for constraint satisfaction after ILP inference
8: end for

```

Although seemingly trivial, the behavior of the ILP solver is highly complex, governed primarily by the logit magnitudes of the dummy predictions. The solver can satisfy a logical implication in two ways: it can correct the final prediction to make the conclusion true, or it can alter a premise so the condition is no longer met, vacuously satisfying the constraint. Naturally, the first scenario is the desired outcome in practice.

To account for this variability, we tested three distinct scenarios. First, we applied a uniform distribution to the conclusion logits. This prompts the solver to correct the conclusion, as it maximizes the global objective at a minimum cost. Second, we heavily biased the conclusion logits toward an incorrect answer while keeping the logit magnitude uniform across the entire chain. In this case, the solver alters one of the premises, as doing so represents a more economical optimization path. Third, we maintained the incorrect bias for the conclusion but deliberately reduced its logit magnitude relative to the premise questions. Because modifying lower-magnitude logits incurs a lower penalty, the solver reverts to correcting the conclusion to satisfy the constraint.

Ultimately, a comprehensive integration test that accounts for these variable scenarios provides high confidence in the constraint implementation. Rather than merely inferring correctness from observed satisfaction rates during training, this methodology offers a deterministic guarantee that the modeled constraints function exactly as intended.

5 Experimental Setup

5.1 Data and Splits

Our experiments were evaluated on the TimeBank-Dense (TB-Dense) dataset ([Cassidy et al., 2014](#)). The dataset’s overall label distribution is characterized by severe class imbalance: while VAGUE accounts for nearly half of all data points, SIMULTANEOUS comprises merely 1.69%.

The standard split commonly used in existing literature is unfortunately compromised by notable distribution shifts across the train, dev, and test sets. Because we use Macro F1 as our primary metric for optimization, training on this default split introduced instability and caused models to overfit to sparse minority-class instances in the validation set.

Table 5.1: Label distribution breakdown across Train, Dev, and Test splits for TB-Dense.

Relation	Train		Dev		Test		Total	
	<i>N</i>	%	<i>N</i>	%	<i>N</i>	%	<i>N</i>	%
Vague	4,237	46.78%	769	45.37%	904	46.08%	5,910	46.48%
Before	1,850	20.42%	347	20.47%	393	20.03%	2,590	20.37%
After	1,486	16.41%	283	16.70%	335	17.07%	2,104	16.55%
Is Included	732	8.08%	158	9.32%	170	8.66%	1,060	8.34%
Includes	603	6.66%	112	6.61%	121	6.17%	836	6.58%
Simultaneous	150	1.66%	26	1.53%	39	1.99%	215	1.69%
Total	9,058	100.00%	1,695	100.00%	1,962	100.00%	12,715	100.00%

To resolve this issue, we generated a custom split designed to enforce consistent label distributions across all subsets. Table 5.1 presents the resulting split and class distributions. To prevent document-level data leakage, we applied document-level stratified K -fold cross-validation. Specifically, we evaluated 100 random initialization seeds and selected the split that minimized the maximum distance between class distributions across the training, validation, and test sets.

5.2 Evaluation Metrics

As our primary performance metric we use Macro F1. While Micro F1 represents the current field standard, it heavily favors majority classes—such as the overrepresented VAGUE class in TB-Dense—disproportionately distorting overall evaluation. By weighting all classes equally, Macro F1 better prioritizes minority class performance and prevents these systemic imbalances.

For measuring constraint satisfaction, we used the metrics provided by the DomiKnowS framework. Total accuracy reflects the average constraint satisfaction percentage across all evaluated DataNode–constraint pairs (Equation 5.1). Here, D denotes the number of DataNodes (batches), while C denotes the number of declared constraints. Furthermore, i and j index the individual DataNodes and constraints, respectively. The term $\text{sat}_{i,j}$ denotes the constraint satisfaction rate of constraint j for DataNode i :

$$\text{TotalAcc} = \frac{\sum_{i=1}^D \sum_{j=1}^C \text{sat}_{i,j}}{D \cdot C}. \quad (5.1)$$

DataNode accuracy measures the percentage of DataNodes that achieve a 100% constraint satisfaction rate:

$$\text{DataNodeAcc} = \frac{\sum_{i=1}^D \sum_{j=1}^C \mathbf{1}[\text{sat}_{i,j} = 100]}{D \cdot C} \cdot 100. \quad (5.2)$$

Here, the indicator function $\mathbf{1}[\cdot]$ takes the value 1 if $\text{sat}_{i,j} = 100$ and 0 otherwise. Intuitively, this metric evaluates constraint satisfaction as a binary outcome: rather than allowing partial satisfaction, a DataNode–constraint pair contributes to the numerator only if it achieves complete (100%) satisfaction. Consequently, DataNodeAcc reflects the proportion of batch–constraint pairs that fully satisfy their imposed logical constraints.

ifL accuracy represents the average percentage of satisfied conclusions across all DataNode–constraint pairs, considering only constraint instances for which the premise is met:

$$\text{IfLAcc} = \frac{\sum_{(i,j) \in \mathcal{V}_{\text{ifL}}} \text{ifLsat}_{i,j}}{|\mathcal{V}_{\text{ifL}}|}. \quad (5.3)$$

The term $\text{ifLsat}_{i,j}$ denotes the percentage of satisfied conclusions of constraint j for DataNode i , while $\mathcal{V}_{\text{ifL}}$ denotes the set of DataNode–constraint pairs for which a valid $\text{ifLsat}_{i,j}$ value exists. Intuitively, this metric isolates instances where the premise is explicitly satisfied, filtering out cases where constraints are trivially satisfied due to an unfulfilled

premise. It measures the isolated conclusion satisfaction rate—for example, in a transitive chain, it evaluates whether the third answer is correctly inferred given that the first two are correct. Consequently, IfLAcc provides a far more informative assessment of logical rule enforcement than TotalAcc or DataNodeAcc, as its score is not inflated by vacuously satisfied constraints.

5.3 Baselines

To isolate the performance gains of our approach, we trained two unconstrained baseline models: one on the original, unaugmented dataset and the other on the augmented dataset. This distinction allows us to separate the effects of data augmentation from the impact of incorporating constraints. Aside from data augmentation, both baseline models use the exact same BERT-base architecture as the constraint model (see Section 3.3). Throughout this thesis, we refer to these baseline models as **Baseline** and **Augmented Train**, respectively.

5.4 Hyperparameters and Training Details

All models were scheduled to train for up to 10 epochs with early stopping (patience of 3). The learning rate α_w for the primal optimizer was selected from $\{1e-5, 2e-5, 3e-5, 5e-5\}$, and weight decay was set to $1e-3$. The dual learning rate α_λ and constraint scaling constant d were tuned via grid search over $\{0.01, 0.03, 0.5\}$ and $\{1, 5, 10\}$, respectively. Warm-up steps for the primal-dual objective followed the schedule proposed by [Nandwani et al. \(2019\)](#). A detailed mathematical description of these parameters within our optimization algorithm is provided in Section 3.2. To ensure the robustness of our results, all configurations were evaluated across three distinct random seeds (0, 1, and 2). To mitigate class imbalance across all models, we weighted the loss function using inverse class frequency, computed via scikit-learn’s `compute_class_weight` (balanced mode).

6 Experiments

In this chapter, we present the experimental results for the baseline, data-augmented, and full-constraint models. We begin with an overview of global performance to establish broad trends and key differences across models. Next, we analyze relation-specific behavior by comparing class-level F1 scores. We then present constraint satisfaction rates for all three configurations. Following this, we conduct a class-specific constraint ablation study to investigate the impact of individual constraints, potential constraint interactions, and the distinct behaviors of inversion versus transitivity constraints. Finally, we evaluate the effectiveness of constraints in a reduced training data setting.

6.1 Main Results

Table 6.1 compares our constraint model with a baseline model and a data augmented model with no constraints. The constraint model outperforms the baseline model by more than 4% and the data augmented model by more than 1.3% in terms of Macro F1. Additionally, the constraint model achieves significantly smaller variance across seeds: The standard deviation is more than 2.5 times smaller compared to baseline and nearly four times smaller compared to the data augmented model. This shows that adding constraints to the model strongly stabilizes training across different random seeds.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
Baseline	52.98	51.99	51.11	52.03	0.94
Augmented Train	55.79	53.50	56.02	55.10	1.39
Augmented Train + Constraints	56.38	56.81	56.08	56.42	0.37

Table 6.1: Macro F1 results across different random seeds (values in %)

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
Baseline	2.117	2.282	2.212	2.204	0.083
Augmented Train	1.351	1.622	1.530	1.501	0.138
Augmented Train + Constraints	1.047	1.068	1.069	1.061	0.012

Table 6.2: Test Loss results across different random seeds

From Table 6.2 it appears that test loss is significantly lower for the constraint model (nearly seven times lower compared to baseline and more than ten times compared to the

data augmented model). This suggests that the constraint model learns a more confident and accurate decision boundary. The reduction in test loss and variance across seeds means also that logical constraints act as a regularizer, stabilizing training and improving generalization on unseen data.

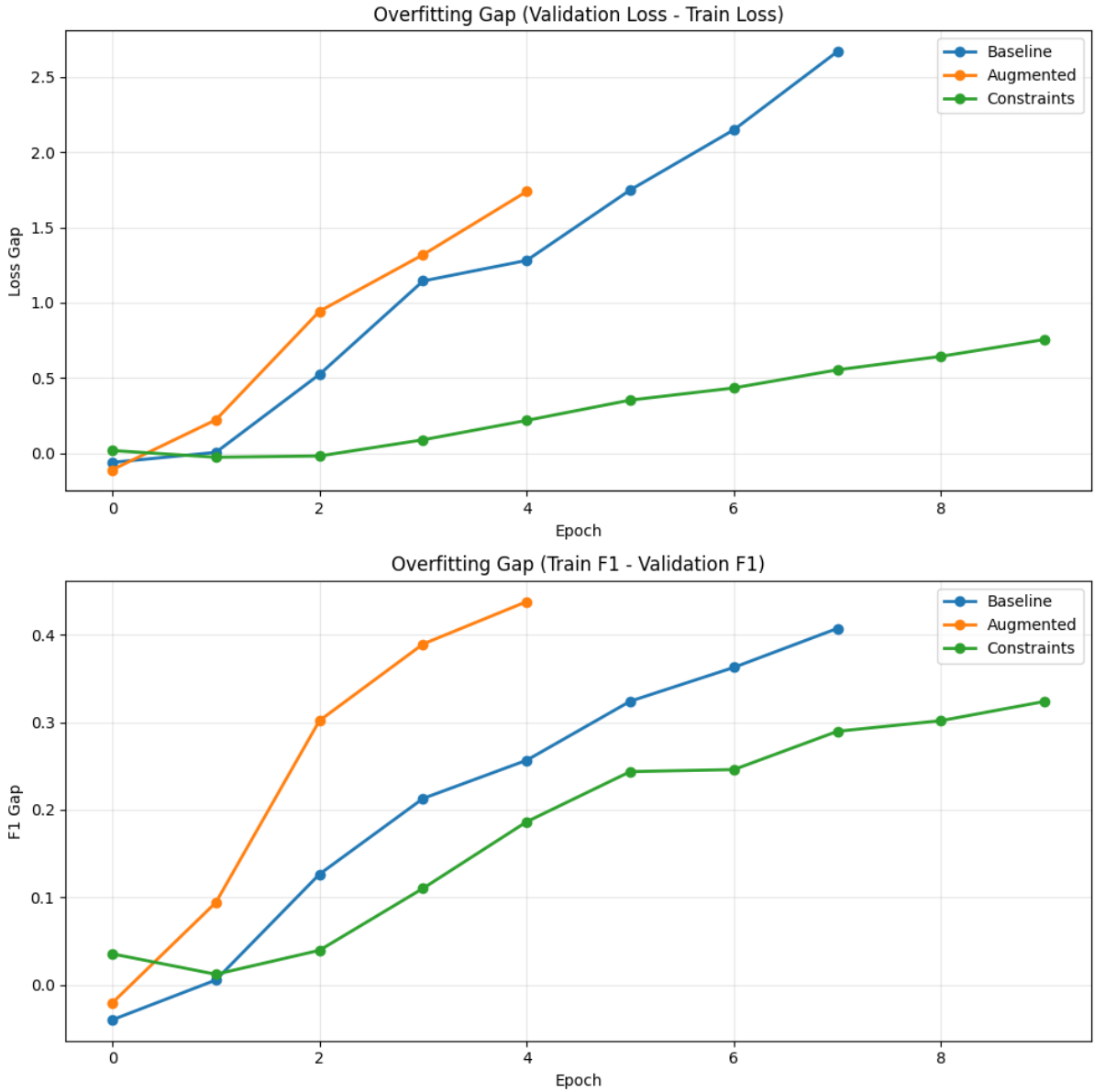


Figure 6.1: Overfitting gap trajectories across training epochs for Loss Gap (top) and F1 Gap (bottom).

To better understand the behavior of the constraint model, we compare its training dynamics with those of the other models. Figure 6.1 shows the difference between training and validation performance across epochs, measured both in terms of loss (top) and F1 score (bottom).

The baseline and data-augmented models display a continuously increasing overfitting gap as training progresses. For both models, the validation loss diverges from the training

loss after the first few epochs, indicating that training performance continues to improve while validation performance stagnates. A similar pattern can be observed for the F1-based overfitting gap, where the difference between training and validation F1 increases consistently over time.

In contrast, the constraint model shows a substantially smaller overfitting gap throughout the training process. While a moderate increase is still present, both the loss-based and F1-based gaps grow a lot more slowly compared to the other two models. Notably, the loss gap of the constraint model remains well below that of the baseline and augmented models across all epochs, and the F1 gap exhibits a flatter trajectory overall.

These results provide further evidence that the logical constraints act as an effective regularizer. Not only do they lead to reduced variance and increased confidence in model predictions, they also reduce the tendency of the model to overfit the training data. This observation is consistent with the lower test loss and reduced variance across random seeds reported in Section 6.1, and supports the interpretation that constraints improve both final performance and training dynamics.

6.2 Relation-Specific Performance

This section investigates whether the improvements introduced by the constraints attribute to consistent improvements across all relation types, or if some relations benefit from constraints more than others. Table 6.3 therefore reports the F1 score for each relation across three different random seeds.

The constraint model achieves the highest average F1 score for four of the six relation types: AFTER, BEFORE, INCLUDES, and IS INCLUDED. The largest improvement can be observed for the AFTER relation, where the constraint model reaches an average F1 of 69.76%, compared to 65.82% for the augmented model and 64.29% for the baseline. For BEFORE, the constraint model achieves 67.65%, compared to 66.34% for the augmented model and 63.51% for the baseline. The IS INCLUDED relation also benefits from the constraints. Its average F1 increases from 50.16% for the baseline to 55.97% with data augmentation and further to 58.97% when constraints are added. For INCLUDES, the effect of the constraints is more moderate in terms of average F1. The constraint model achieves 48.93%, compared to 47.23% for the baseline and 46.49% for the augmented model. These results suggest that the constraints are particularly effective for the directional temporal relations.

However, the effect of the constraints is not uniformly positive across all relations. For SIMULTANEOUS, data augmentation performs best, reaching an average F1 of 32.57%, while the constraint model achieves 29.40%. Nevertheless, the constraint model still substantially outperforms the baseline, which achieves only 20.21%. This indicates that the

Relation	Configuration	Seed 0	Seed 1	Seed 2	Avg	Sd
After	Baseline	64.86	62.03	65.99	64.29	2.04
	Augmented Train	65.43	67.30	64.72	65.82	1.33
	Augmented Train + Constraints	68.84	71.59	68.84	69.76	1.59
Before	Baseline	65.00	63.34	62.20	63.51	1.41
	Augmented Train	65.45	64.91	68.66	66.34	2.03
	Augmented Train + Constraints	68.48	68.23	66.24	67.65	1.23
Includes	Baseline	44.44	47.49	49.76	47.23	2.67
	Augmented Train	52.49	43.09	43.90	46.49	5.21
	Augmented Train + Constraints	50.38	50.00	46.40	48.93	2.20
Is included	Baseline	54.15	47.65	48.67	50.16	3.50
	Augmented Train	57.62	52.32	57.98	55.97	3.17
	Augmented Train + Constraints	57.75	58.66	60.49	58.97	1.40
Simultaneous	Baseline	22.73	23.26	14.63	20.21	4.84
	Augmented Train	33.68	30.00	34.04	32.57	2.24
	Augmented Train + Constraints	30.43	26.67	31.11	29.40	2.39
Vague	Baseline	66.70	68.15	65.44	66.76	1.36
	Augmented Train	60.09	63.38	66.82	63.43	3.37
	Augmented Train + Constraints	62.37	65.75	63.40	63.84	1.73

Table 6.3: F1 scores per class relation across different random seeds (values in %)

constraints retain a positive effect relative to the original model, but do not provide an additional benefit beyond data augmentation for this particular relation. A similar pattern can be observed for VAGUE. The baseline achieves the highest average F1 of 66.76%, while the augmented and constraint models achieve 63.43% and 63.84%, respectively. Thus, neither data augmentation nor the constraints improve performance for this relation.

An additional observation is that the constraints generally reduce the variability across random seeds for the individual relations. The constraint model has the lowest standard deviation for BEFORE, INCLUDES, and IS INCLUDED, and remains considerably more stable than the augmented model for VAGUE. This is consistent with the global results and provides further evidence that the constraints act as a stabilizing mechanism during training.

Overall, the class-specific results indicate that the benefits of the constraints depend on the relation type. The strongest improvements are observed for the four directional temporal relations. In contrast, SIMULTANEOUS and VAGUE do not benefit from the constraints in terms of average F1. This suggests that the global improvement reported in Table 6.1 is driven by substantial gains in specific relation classes rather than by a uniform improvement across all relations. Furthermore, the constraint model achieves a lower standard deviation across seeds for the majority of relations, reinforcing the global trend of enhanced training

stability.

6.3 Constraint Satisfaction

In this section, we evaluate the constraint satisfaction of the baseline, data-augmented, and constraint models using various evaluation metrics.

Metric	Configuration	Seed 0	Seed 1	Seed 2	Avg	Sd
Datanode Accuracy	Baseline	98.67	98.79	98.98	98.81	0.16
	Augmented Train	98.62	98.98	98.59	98.73	0.22
	Augmented Train + Constraints	99.85	99.78	99.68	99.77	0.09
Total Accuracy	Baseline	99.16	99.17	99.38	99.24	0.12
	Augmented Train	99.00	99.35	99.10	99.15	0.18
	Augmented Train + Constraints	99.88	99.86	99.76	99.83	0.06
IfL Accuracy	Baseline	83.64	80.56	85.41	83.20	2.45
	Augmented Train	82.91	85.54	83.90	84.12	1.33
	Augmented Train + Constraints	97.12	96.12	94.08	95.77	1.55

Table 6.4: Constraint evaluation metrics across different random seeds (values in %).

Table 6.4 summarizes the performance of the three constraint evaluation metrics evaluated across different random seeds. For a detailed explanation of the metrics see 5.2.

Across all metrics, the constraint model consistently outperforms the other configurations. Datanode accuracy improves by roughly 1% with the constraint model, while Total accuracy increases by over 0.5% compared to the non-constraint baselines. Most notably, IfL accuracy reaches an impressive 95.77%—an improvement of over 11 percentage points relative to the other models. This reduces the conditional implication error rate from approximately 16% down to 4.23%, representing a 73.6% reduction in implication violations for arguably the most vital constraint satisfaction metric.

Overall, baseline Datanode accuracy and Total accuracy remain high because many constraint instances are trivially or automatically satisfied—including implication instances with false premises. The IfL metric removes this effect by evaluating only instances where the premise holds true, making it far more sensitive to whether the model actually enforces the required conclusions. Consequently, IfL accuracy serves as the most meaningful metric when evaluating constraint satisfaction.

In conclusion, incorporating constraints during training significantly outperforms the baseline configurations in logical constraint satisfaction.

6.4 Constraints Ablation Study

6.4.1 Relation-Specific Constraint Analysis

We consider two settings for relation-specific constraint analysis. In the first setting, we remove all constraints involving a specific relation type from the constraint graph while leaving the remaining constraints unchanged. For example, when analyzing the relation AFTER, we disable all constraints involving AFTER and evaluate the resulting model. This allows us to investigate how the removal of constraints associated with a particular relation affects the overall system.

In the second setting, we enable only the constraints associated with a single relation type and disable all other constraints. For example, when analyzing AFTER, only the constraints involving AFTER are enabled, while constraints associated with all other relations are disabled. This setting allows us to examine the effect of the relation-specific constraints in isolation.

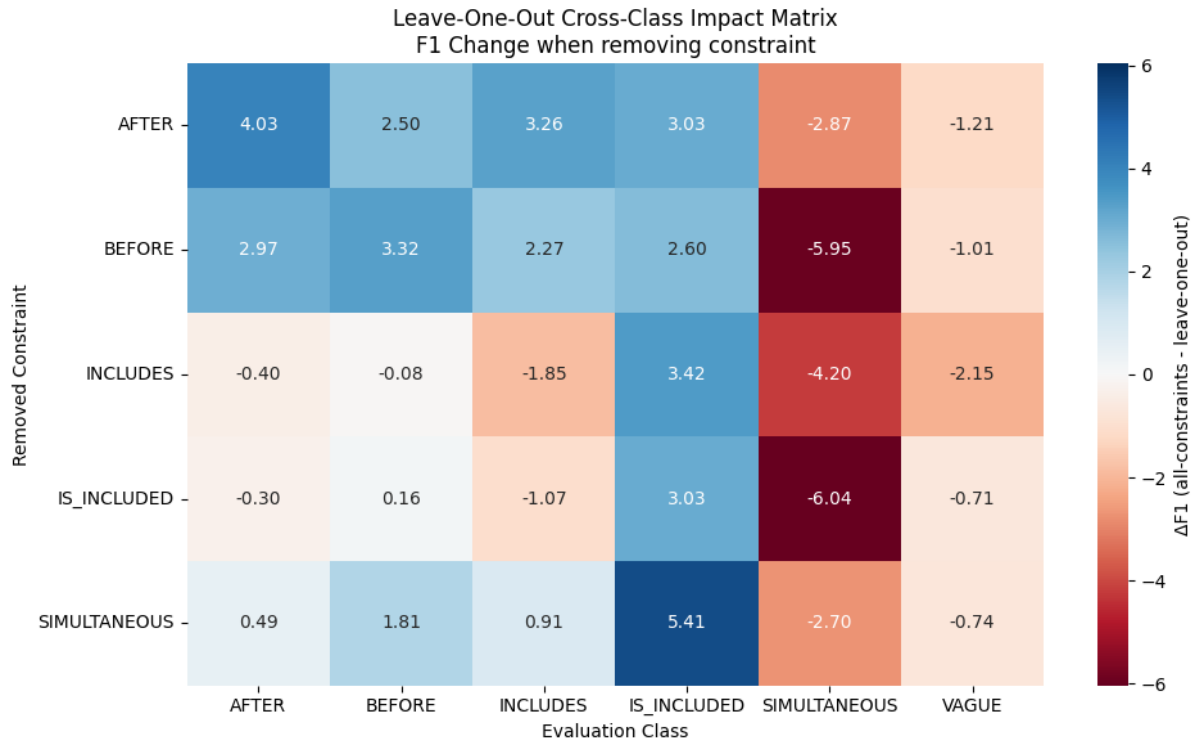


Figure 6.2: Leave-One-Out Cross-Class Impact Matrix. Entries display the F1 score difference between the all-constraints model and the leave-one-out model ($F1_{\text{all}} - F1_{\text{leave-one-out}}$). Blue values indicate that the all-constraints model achieves higher performance, meaning constraint removal degrades performance. Red values indicate that the leave-one-out model performs better on that class.

Figure 6.2 shows the Leave-One-Out Cross-Class Impact Matrix. Each row corresponds to a model trained with all constraints except the constraints of one relation type, while

each column represents the relation-specific performance. Each matrix entry shows the difference between the relation-specific F1 score of the all-constraints model and that of the model with the corresponding relation-specific constraints removed. Blue entries indicate that the all-constraints model achieves a higher F1 score, meaning that removing the corresponding relation-specific constraints leads to worse performance. Conversely, red entries indicate that the model with the relation-specific constraints removed achieves a higher F1 score than the all-constraints model, suggesting that removing those constraints improves performance for the respective relation.

For AFTER, BEFORE and SIMULTANEOUS we observe that the all-constraints model performs better for all relations except SIMULTANEOUS and VAGUE. This suggests that, overall, including constraints specific to AFTER and BEFORE in the constraint system has a largely positive effect. For the INCLUDES and IS INCLUDED models, we observe no clear positive or negative effect.

Furthermore, from a relation-specific perspective, SIMULTANEOUS and VAGUE consistently benefit from constraint removal. This observation is in line with the results presented in Table 6.3, where the data-augmented model outperforms the constraints model for both SIMULTANEOUS and VAGUE. This suggests that, for these two relations, the additional constraints may restrict the model’s ability to make correct predictions rather than providing beneficial information.

Inspecting these trends, one might ask whether a model with a specific type of constraint removed could outperform an all-constraints model. Comparing the macro F1 scores of the leave-one-out models with that of the all-constraints model, we find that both the INCLUDES and IS INCLUDED models achieve an F1 score that is 0.58 percentage points higher than that of the all-constraints model (all-constraints model achieves an F1 score of 56.42%).

To better understand the effects of relation-specific constraints in isolation, we also trained models in which constraints are enabled for only a single relation type. Figure 6.3 shows the Only-One Cross-Class Impact Matrix. Each row corresponds to a model trained with constraints associated with only one specific relation type, while the columns represent the relation-specific performance. Each matrix entry shows the difference between the relation-specific F1 score of the isolation model and that of the no-constraints model. Blue entries indicate that the isolation model achieves a higher F1 score for the respective relation, while red entries indicate that the no-constraints model performs better.

For the AFTER isolation model, we observe red values across the entire row. This is caused by one seed run in which the model predicted only VAGUE, resulting in very low average class F1 scores and consequently explaining the red entries. This suggests that the AFTER-only configuration may not provide a robust model configuration, as its performance appears sensitive to the random seed. For the BEFORE isolation model, the no-constraints model outperforms it for the majority of relations, likewise indicating that BEFORE constraints alone are insufficient. In contrast, the INCLUDES constraint has a positive

effect for all relations except BEFORE, making it a promising candidate for use in isolation. The IS INCLUDED and SIMULTANEOUS constraints also yield a small positive effect in isolation.

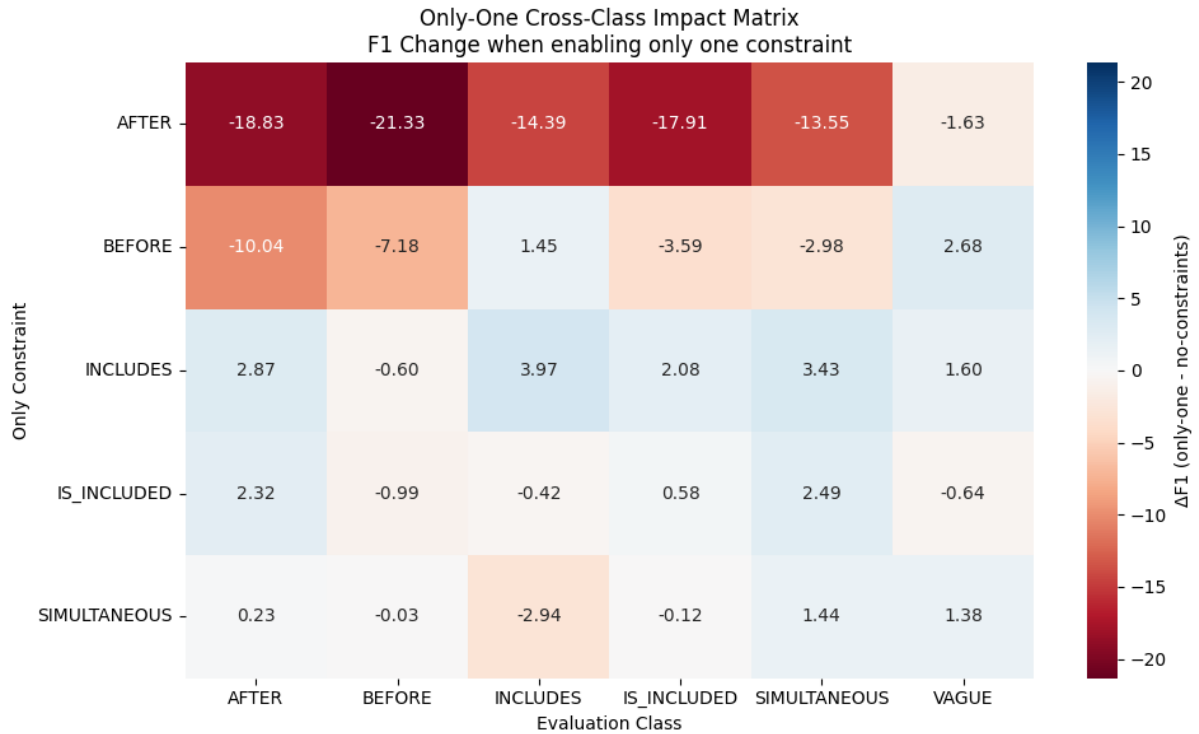


Figure 6.3: Only-One Cross-Class Impact Matrix. Entries display the F1 score difference between the isolation constraints model and the no-constraints model ($F1_{\text{isolation}} - F1_{\text{no-constraints}}$). Blue values indicate that enabling the isolated constraint improves performance over the no-constraints model. Red values indicate that the no-constraints model outperforms the isolated constraints model, meaning that the isolated constraint has a negative effect on performance.

Combining the insights from both ablation studies, we can draw the following conclusions about relation-specific constraints. The AFTER constraint is insufficient on its own but has a positive effect when combined with the other constraints. The same pattern is observed for BEFORE. In contrast, the INCLUDES constraint does not consistently improve performance when all constraints are enabled, suggesting potential interactions or conflicts with other constraints. However, it has a strong positive effect in isolation, indicating that its overall behavior is governed by more complex interaction effects. The IS INCLUDED constraint has a particularly strong impact on the performance of SIMULTANEOUS. Although its effect in isolation is relatively small, its strong interaction with the other constraints suggests that it plays an important role in the overall constraint system. Similarly, the SIMULTANEOUS constraint has only a small effect in isolation but improves performance when combined with the other constraints, highlighting a positive interaction effect. Interestingly, removing the SIMULTANEOUS constraint improves performance on SIMULTANEOUS itself, suggesting a potential conflict between this constraint and other components of the system.

Overall, the ablation studies reveal complex interaction effects between relation-specific constraints. In particular, the results demonstrate that the effect of a relation-specific constraint cannot be evaluated independently of the other constraints present in the system. Instead, the constraints form an interconnected system in which interactions across relations can either improve performance or negatively affect specific relations. Moreover, removing a constraint may improve the performance of an individual relation while providing little or no benefit to the overall system. Such a modification may also negatively affect overall constraint satisfaction. Therefore, relation-specific performance gains should be considered alongside their broader effects on system-wide performance and constraint satisfaction.

6.4.2 Inverse vs Transitive Constraints

While the previous section provided insights into relation-specific constraint effects and interactions, this section focuses on the differences in task performance between inverse and transitive constraints.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
Inverse	54.58	55.10	57.06	55.58	1.31
Transitive	56.45	52.87	55.26	54.86	1.82

Table 6.5: Macro F1 results across different random seeds (values in %)

To better understand the differences and effects of inverse and transitive constraints, a model with only inverse constraints enabled and another with only transitive constraints enabled were trained. Table 6.5 presents their performance on the test set. It can be observed that enabling either constraint type in isolation does not improve overall model performance compared to a model trained with all constraints.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
Baseline	79.79	78.21	82.67	80.22	2.26
Augmented Train	77.08	82.73	78.48	79.43	2.94
Augmented Train + Constraints	100.00	100.00	100.00	100.00	0.00

Table 6.6: ifL Inverse Accuracy results across different random seeds (values in %)

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
Baseline	88.85	84.38	90.00	87.74	2.97
Augmented Train	91.94	90.50	93.53	91.99	1.52
Augmented Train + Constraints	94.44	92.48	89.04	91.99	2.73

Table 6.7: ifL Transitive Accuracy results across different random seeds (values in %)

While understanding the isolated effects of inverse and transitive constraints on overall performance is useful, their effects on constraint satisfaction are arguably more important. To gain insights into the differences between inverse and transitive constraints in terms of satisfaction rate, we evaluated the baseline, the data-augmented model, and the full-constraints model separately using ifL inverse accuracy and ifL transitive accuracy.

Table 6.6 provides an overview of the different models evaluated using ifL inverse accuracy. We can observe that the full-constraints model achieves perfect deterministic performance on inverse constraints, with a performance gap of roughly 20 percentage points compared to both the baseline and the data-augmented model.

In contrast to the substantial performance gap observed for inverse constraint satisfaction, Table 6.7 surprisingly shows that the data-augmented model and the full-constraints model achieve the same performance in terms of ifL transitive accuracy, with an accuracy of 91.99%. This corresponds to an improvement of roughly 4 percentage points over the baseline. These results suggest that the baseline and data-augmented models are already able to learn transitive constraints effectively from supervision alone.

6.5 Constraint Effects on Reduced Training Data

After gaining an understanding of relation-specific constraint dynamics and the differences between inverse and transitive constraints, we investigate how constraints affect task performance and constraint satisfaction in a reduced-training-data setting. Therefore, we trained a baseline model without constraints and a full-constraints model using 50% of the training data. To ensure that the two models were trained on the same data, we used the same data-augmented training set for both models, meaning that the constraint chains were present in the training data of both the constraints and no-constraints model.

Table 6.8 presents the results on the test set. The full-constraints model achieves a macro F1 score of 55.62%, outperforming the no-constraints model by approximately four percentage points. Furthermore, the full-constraints model trained on only 50% of the training data slightly outperforms the data-augmented model trained on the full training set by 0.5 percentage points.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
No Constraints	52.10	49.26	53.32	51.56	2.08
Constraints	54.17	56.33	56.37	55.62	1.26

Table 6.8: Macro F1 results using 50% training data (values in %)

The difference between the no-constraints and full-constraints models is even more pronounced for constraint satisfaction. As shown in Table 6.9, the full-constraints model achieves an ifL accuracy of 90.74%, compared to 67.13% for the no-constraints model,

corresponding to a performance gap of approximately 23 percentage points. For comparison, the gap in the full-data setting is 11 percentage points, meaning that the difference between the two models increases by approximately 50% when the training data is reduced to 50%.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
No Constraints	65.19	63.53	72.67	67.13	4.87
Constraints	93.37	88.45	90.40	90.74	2.48

Table 6.9: ifL Accuracy results using 50% training data (values in %)

Building on the results presented in Section 6.4.2, we further investigate the differences in ifL accuracy for inverse and transitive constraints in the 50% training-data setting. Table 6.10 shows an even larger performance gap between the no-constraints and full-constraints models for inverse constraints, with a difference of approximately 34 percentage points in favor of the constrained model. Compared to the full-data setting, the full-constraints model loses approximately 6 percentage points in inverse ifL accuracy, whereas the no-constraints model loses approximately 13 percentage points. Consequently, reducing the training data from 100% to 50% increases the gap between the two models from approximately 20 to 34 percentage points.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
No Constraints	54.76	59.10	67.06	60.31	6.24
Constraints	98.39	88.89	95.83	94.37	4.92

Table 6.10: ifL Inverse Accuracy results using 50% training data (values in %)

While the data-augmented and full-constraints models achieved the same transitive ifL accuracy in the full-data setting, the results differ in the 50% data setting (Table 6.11). The full-constraints model loses only approximately three to four percentage points compared to its full-data performance, whereas the no-constraints model loses approximately 13 percentage points. As a result, the gap between the two models increases to approximately nine percentage points in the reduced-data setting.

Method	Seed 0	Seed 1	Seed 2	Avg	Sd
No Constraints	83.94	72.53	81.64	79.37	6.03
Constraints	90.38	88.14	87.86	88.79	1.38

Table 6.11: ifL Transitive Accuracy results using 50% training data (values in %)

To sum up, reducing the training data to 50% increases the performance gap between the full-constraints and no-constraints models. In terms of task performance, the full-constraints model trained on only 50% of the data performs slightly better than the data-augmented model trained on the full dataset. The effect is even more pronounced for constraint satisfaction: the gap between the constrained and unconstrained models increases

substantially in the reduced-data setting. At the same time, the previously observed difference between inverse and transitive constraints remains consistent, with inverse constraints being substantially harder to learn without explicit constraints than transitive constraints.

7 Discussion

In this work, we investigated the effects of integrating inverse and transitivity constraints during model training. We found positive effects not only on task performance and logical consistency, but also on the entire training process.

In addition to producing an improved Macro F1 score compared to baselines, the primal-dual objective acts as an effective regularizer, leading to reduced overfitting and lower variance across random seeds. It should be noted that the constraint model outperforms two different baselines: one is trained on the same augmented dataset as the constraint model but with no constraints, while the other baseline is trained with no augmented data and no constraints. Because of this design choice, we can better identify whether improvements are due to data augmentation or are the consequence of including constraints. For task performance, it appears most of the performance gain is the result of data augmentation. For training dynamics, data augmentation alone produces more overfitting and higher variance across seeds compared to the baseline with no augmented training data. The constraint model, on the other hand, makes more confident predictions, has considerably lower variance, and overfits significantly less, highlighting the regularization potential of our approach. Finally, a direct comparison of our task performance with prior works on TB-Dense is not feasible, as we opted for a custom train-test split to mitigate the severe training instability observed with the standard split.

Regarding relation-specific performance, we found that the constraint model outperformed the baselines on four of the six relations. The baselines achieved higher scores only on the `SIMULTANEOUS` and `VAGUE` relations. Interestingly, many prior works report an F1 score of zero for `SIMULTANEOUS`, as it represents the most extreme minority category in TB-Dense. Our literature review identified only two recent works achieving F1 scores of approximately 7% and 36% on this relation (Hu et al., 2024; Zhang, Zang, Liu, Wei, & Hu, 2024). Although we utilize a custom train-test split, the F1 scores of 29.4% for our constraint model and 32.57% for our data-augmented model mark a notable achievement given the historical difficulty of predicting this class. These performance gains can be primarily attributed to three design choices: the use of Macro F1 rather than Micro F1 as the primary training objective, the application of data augmentation, and the utilization of class weights.

Furthermore, our constraint ablation study revealed complex interaction effects among the different temporal rules. It follows that a specific constraint cannot be evaluated in iso-

lation, as it operates within an interconnected system characterized by both positive and negative cross-relation interactions. These findings highlight a potential trade-off when deciding to enable or disable certain constraints based on relation type: while removing a specific rule might boost the classification performance of an individual relation, it can degrade the accuracy and logical consistency of the overall system. Additionally, it is highly questionable whether such a trade-off justifies the computational overhead of the extensive hyperparameter tuning it requires. Direct comparisons with prior literature are not possible here, as we are, to the best of our knowledge, the first to conduct a relation-specific constraint analysis on the TB-Dense corpus and within the broader field of TRE.

Considering overall constraint satisfaction, our model significantly outperforms the unconstrained baselines. The starkest contrast emerges when comparing constraint types: while the primal-dual objective enables perfect logical consistency for inverse constraints, both the constraint model and the data-augmented baseline perform identically on transitivity. This suggests that while transitivity can be learned effectively from the data alone (provided the training set is sufficiently large), inverse constraints are significantly more difficult for the model to learn without explicit logical guidance.

When positioning these results within the existing literature, it is notable that very few studies report detailed constraint satisfaction metrics. One exception is [Hwang et al. \(2022\)](#), who, when implementing the soft-logic approach of [H. Wang et al. \(2020\)](#) as a baseline, reported an inverse constraint violation rate of 22–35% across various benchmarks. This strongly indicates that our primal-dual objective, which learns individual weights for each constraint, enforces symmetry much more effectively than soft-logic methods relying on a static penalty parameter. A definitive conclusion would require a direct comparison on identical benchmarks. Furthermore, while the box embedding architecture of [Hwang et al. \(2022\)](#) guarantees zero inverse violations by design, they rely on a static constraint loss term to improve transitivity, which ultimately compromises their overall task performance. In contrast, our primal-dual approach matches their perfect inverse consistency through optimization alone while maintaining high task performance. For transitivity, our approach yields highly competitive results, though it remains short of perfect consistency. If absolute transitive consistency is strictly required for a downstream application, an additional ILP post-processing step could easily be applied to our model’s output.

Inspired by the experiments of [Nandwani et al. \(2019\)](#), we conducted an analysis in a reduced training data setting. Therefore, we trained a baseline and a constraint model on 50% of the available data. First, we found that the task performance gap widens in favor of the constraint model. Remarkably, the constraint model trained on 50% of the data matches the performance of the data-augmented baseline trained on the full dataset. Furthermore, the gap in the constraint satisfaction rate between the constrained and unconstrained models increases significantly in the reduced-data setting. Consequently, the constraint model produces significantly fewer transitivity violations. The observed difference between in-

verse and transitive constraints from the full-data setting remains consistent. [Nandwani et al. \(2019\)](#) conclude from their experiments that the benefit of constraint learning decreases as training data increases, and our results confirm this trend for TRE when learning constraints via a primal-dual objective. Crucially, while our constraint learning approach led to significant improvements in the full-data setting, these benefits become even more pronounced when data is limited.

Finally, we acknowledge several limitations in our research design. First, our experiments were evaluated solely on the TB-Dense dataset, which may limit generalizability. Future work could validate these findings across other benchmarks, such as MATRES and TD-Discourse. Second, our use of a custom train-test split—while methodologically necessary—precludes direct empirical comparisons with existing state-of-the-art baselines. Third, due to computational constraints, our results are averaged over only three random seeds, and BERT is the sole encoder architecture evaluated. To ensure broader statistical robustness and generalizability, future research should assess additional architectures across a larger set of random seeds.

8 Conclusion

The primary objective of this thesis was to uncover the precise effects of integrating temporal rules, such as inversion and transitivity, via differentiable soft logic during model training for the TRE task. To achieve this, we fine-tuned a BERT model using a primal-dual objective that dynamically learns individual weights for each modeled constraint.

We demonstrated the regularizing effects of applying inverse and transitivity constraints: they significantly reduce overfitting and variance across random seeds while simultaneously improving overall task performance. Our ablation study, which evaluated constraints from a relation-specific perspective, revealed complex positive and negative cross-relation interactions. Furthermore, our analysis of constraint satisfaction rates showed that while transitivity can be learned from the data alone given enough training examples, inverse constraints require explicit logical guidance. Finally, we established that a constrained model trained on just 50% of the available data matches the performance of an unconstrained model trained on the full dataset.

Our approach successfully combines the inductive power of learning from data with the structural reliability provided by explicit logical constraints. It retains the flexibility of supervised learning while approaching the absolute logical consistency of ILP-based methods, effectively eliminating the historical trade-off between task performance and logical consistency. Moreover, it requires significantly less manually annotated training data to achieve highly competitive performance.

It should be noted that further evaluation on additional benchmarks, utilizing diverse architectures and a wider range of random seeds, is necessary to fully cement the potential of modeling constraints via a primal-dual objective for TRE.

Beyond confirming the generalizability of our results, future research should focus on expanding the scope of application. Transitioning from local pairwise classification to the global prediction of entire temporal documents—by leveraging the extensive pre-trained knowledge and expanded context windows of the latest generation of LLMs—represents a vital next step in verifying the scalability of this approach.

Nevertheless, as long as smaller encoder models continue to rival the performance of LLMs in the field of TRE, our method remains a highly effective solution for achieving strong task performance coupled with strict logical consistency, particularly in low-resource settings where annotated training data is scarce.

List of Figures

3.1	BERT Architecture	29
4.1	DomiKnowS architecture	35
4.2	DomiKnowS graph definition	37
6.1	Overfitting Gap	46
6.2	Leave-One-Out Cross-Class Impact Matrix	50
6.3	Only-One Cross-Class Impact Matrix	52

List of Tables

2.1	Allen's 13 Interval Relations	15
3.1	Temporal relation definitions	25
3.2	Inverse mappings	26
3.3	Transitive composition rules	26
5.1	Label distribution TB-Dense	41
6.1	Main results Macro F1	45
6.2	Main results Test Loss	45
6.3	Class F1 scores	48
6.4	Constraint evaluation metrics	49
6.5	Macro F1 results for Inverse and Transitive Models	53
6.6	ifL Inverse Accuracy	53
6.7	ifl Transitive Accuracy	53
6.8	Macro F1 results using 50% training data	54
6.9	ifL Accuracy results using 50% training data	55
6.10	ifL Inverse Accuracy results using 50% training data	55
6.11	ifL Transitive Accuracy results using 50% training data	55

List of Algorithms

1	Primal-Dual Optimization Algorithm	28
2	Data Augmentation Algorithm	31
3	Question Batching Algorithm	32
4	Integration Test for Temporal Constraint Graph	39

Bibliography

- Abend, O., Cohen, S. B., & Steedman, M. (2015). Lexical event ordering with an edge-factored model. In *Proceedings of the 2015 conference of the north american chapter of the association for computational linguistics: Human language technologies* (pp. 1161–1171). <https://doi.org/10.3115/v1/N15-1122>
- Alfattni, G., Peek, N., & Nenadic, G. (2020). Extraction of temporal relations from clinical free text: A systematic review of current approaches. *Journal of Biomedical Informatics*, 108, 103488. <https://doi.org/10.1016/j.jbi.2020.103488>
- Allen, J. F. (1983). Maintaining knowledge about temporal intervals. *Commun. ACM*, 26(11), 832–843. <https://doi.org/10.1145/182.358434>
- Bethard, S. J. (2007). *Finding event, temporal and causal structure in text: A machine learning approach* (Ph.D.). University of Colorado at Boulder, United States - Colorado.
- Bramsen, P., Deshpande, P., Lee, Y. K., & Barzilay, R. (2006a). Finding temporal order in discharge summaries. In *AMIA annual symposium proceedings* (Vol. 2006, p. 81). <https://pmc.ncbi.nlm.nih.gov/articles/PMC1839632/>
- Bramsen, P., Deshpande, P., Lee, Y. K., & Barzilay, R. (2006b). Inducing temporal graphs. In *Proceedings of the 2006 conference on empirical methods in natural language processing* (pp. 189–198). <https://aclanthology.org/W06-1623/>
- Cassidy, T., McDowell, B., Chambers, N., & Bethard, S. (2014). An Annotation Framework for Dense Event Ordering. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 501–506). Baltimore, Maryland: Association for Computational Linguistics. <https://doi.org/10.3115/v1/P14-2082>
- Chambers, N., Cassidy, T., McDowell, B., & Bethard, S. (2014). Dense event ordering with a multi-pass architecture. *Transactions of the Association for Computational Linguistics*, 2, 273–284. https://doi.org/10.1162/tacl_a_00182
- Chambers, N., Wang, S., & Jurafsky, D. (2007). Classifying temporal relations between events. In *Proceedings of the 45th annual meeting of the association for computational linguistics companion volume proceedings of the demo and poster sessions* (pp. 173–176). <https://aclanthology.org/P07-2044/>
- Cheng, Y., Anick, P., Hong, P., & Xue, N. (2013). Temporal relation discovery between events and temporal expressions identified in clinical narrative. *Journal of biomedical*

- informatics*, 46, 48–53. <https://doi.org/10.1016/j.jbi.2013.09.010>
- Denis, P., & Muller, P. (2011). Predicting globally-coherent temporal structures from texts via endpoint inference and graph decomposition. In *IJCAI-11-International Joint Conference on Artificial Intelligence*. <https://inria.hal.science/inria-00614765/>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North* (pp. 4171–4186). Minneapolis, Minnesota: Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Dligach, D., Miller, T., Lin, C., Bethard, S., & Savova, G. (2017). Neural Temporal Relation Extraction. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers* (pp. 746–751). Valencia, Spain: Association for Computational Linguistics. <https://aclanthology.org/E17-2118/>
- Eirew, A., Bar, K., & Dagan, I. (2025). Beyond Pairwise: Global Zero-shot Temporal Graph Generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 31440–31458). Suzhou, China: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.emnlp-main.1601>
- Fatemi, B., Kazemi, S. M., Tsitsulin, A., Malkan, K., Yim, J., Palowitch, J., ... Perozzi, B. (2025). Test of time: A benchmark for evaluating llms on temporal reasoning. In *International Conference on Learning Representations* (Vol. 2025, pp. 94426–94447). https://proceedings.iclr.cc/paper_files/paper/2025/hash/eb7295a8bc613b375726659c2ecd6f14-Abstract-Conference.html
- Ferro, L., Mani, I., Sundheim, B., & Wilson, G. (2001). Tides temporal annotation guidelines version 1.0. 2. *The MITRE Corporation, McLean-VG-USA*.
- Gupta, N. (2015). *Temporal Information Extraction Extracting Events and Temporal Expressions A Literature Survey* (Tech. Rep.). https://www.cfilt.iitb.ac.in/~cfiltnew/resources/surveys/Temporal_Information_Extraction-Naman-June15.pdf
- Han, R., Hsu, I.-H., Yang, M., Galstyan, A., Weischedel, R., & Peng, N. (2019). Deep Structured Neural Network for Event Temporal Relation Extraction. *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, 666–106. <https://doi.org/10.18653/v1/K19-1062>
- Han, R., Ning, Q., & Peng, N. (2019). Joint Event and Temporal Relation Extraction with Shared Representations and Structured Prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 434–444). Hong Kong, China: Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1041>

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Homburger, O., & Bar, K. (2025). Large Temporal Models: Unlocking Temporal Understanding in LLMs for Temporal Relation Classification. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics* (pp. 2156–2171). Mumbai, India: The Asian Federation of Natural Language Processing and The Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.ijcnlp-long.117>
- Hu, Y., Huang, Q., & Feng, Y. (2024, August). *Only One Relation Possible? Modeling the Ambiguity in Event Temporal Relation Extraction*. arXiv.
- Hwang, E., Lee, J.-Y., Yang, T., Patel, D., Zhang, D., & McCallum, A. (2022). Event-Event Relation Extraction using Probabilistic Box Embedding. In S. Muresan, P. Nakov, & A. Villavicencio (Eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 235–244). Dublin, Ireland: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.acl-short.26>
- Kougia, V., Sedova, A., Stephan, A. J., Zaporozets, K., & Roth, B. (2024). Analysing zero-shot temporal relation extraction on clinical notes using temporal consistency. In *Proceedings of the 23rd Workshop on Biomedical Natural Language Processing* (pp. 72–84). Bangkok, Thailand: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.bionlp-1.6>
- Laokulrat, N., Miwa, M., & Tsuruoka, Y. (2014). Exploiting timegraphs in temporal relation classification. In *Proceedings of TextGraphs-9: the workshop on Graph-based Methods for Natural Language Processing* (pp. 6–14). <https://doi.org/10.3115/v1/W14-3702>
- Laokulrat, N., Miwa, M., & Tsuruoka, Y. (2016). Stacking approach to temporal relation classification with temporal inference. *Information and Media Technologies*, 11, 53–78. <https://doi.org/10.11185/imt.11.53>
- LeCun, Y., & Bengio, Y. (1998). Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*. <https://hal.science/hal-05083427/>
- Leeuwenberg, A., & Moens, M.-F. (2019, September). A Survey on Temporal Reasoning for Temporal Information Extraction from Text. *Journal of Artificial Intelligence Research*, 66, 341–380. <https://doi.org/10.1613/jair.1.11727>
- Lim, C.-G., Jeong, Y.-S., & Choi, H.-J. (2019). Survey of Temporal Information Extraction. *Journal of Information Processing Systems*, 15(4), 931–956. <https://jips-k.org/digital-library/manuscript/file/23046/JIPS-2019-15-4-931.pdf>
- Liu, J., Xu, J., Chen, Y., & Zhang, Y. (2021, August). Discourse-Level Event Temporal

- Ordering with Uncertainty-Guided Graph Completion. In (Vol. 4, pp. 3871–3877). <https://doi.org/10.24963/ijcai.2021/533>
- Mani, I., Verhagen, M., Wellner, B., Lee, C. M., & Pustejovsky, J. (2006). Machine Learning of Temporal Relations. In N. Calzolari, C. Cardie, & P. Isabelle (Eds.), *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics* (pp. 753–760). Sydney, Australia: Association for Computational Linguistics. <https://doi.org/10.3115/1220175.1220270>
- Meng, Y., & Rumshisky, A. (2018). Context-aware neural model for temporal information extraction. In *Proceedings of the 56th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 527–536). <https://doi.org/10.18653/v1/P18-1049>
- Meng, Y., Rumshisky, A., & Romanov, A. (2017). Temporal Information Extraction for Question Answering Using Syntactic Dependencies in an LSTM-based Architecture. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 887–896. (Conference Name: Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing) <https://doi.org/10.18653/v1/D17-1092>
- Nandwani, Y., Pathak, A., Singla, M., & Singla, P. (2019). A Primal Dual Formulation For Deep Learning With Constraints. In *Advances in Neural Information Processing Systems* (Vol. 32). https://proceedings.neurips.cc/paper_files/paper/2019/hash/cf708fc1decf0337aded484f8f4519ae-Abstract.html
- Ning, Q., Feng, Z., Wu, H., & Roth, D. (2018). Joint Reasoning for Temporal and Causal Relations. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 2278–2288). Melbourne, Australia: Association for Computational Linguistics. <https://doi.org/10.18653/v1/P18-1212>
- Olex, A. L., & McInnes, B. T. (2021, June). Review of Temporal Reasoning in the Clinical Domain for Timeline Extraction: Where we are and where we need to be. *Journal of Biomedical Informatics*, 118, 103784. <https://doi.org/10.1016/j.jbi.2021.103784>
- Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532–1543). <https://doi.org/10.3115/v1/D14-1162>
- Premisri, T., & Kordjamshidi, P. (2025, April). Neuro-symbolic Training for Reasoning over Spatial Language. In L. Chiruzzo, A. Ritter, & L. Wang (Eds.), *Findings of the Association for Computational Linguistics: NAACL 2025* (pp. 2395–2414). Albuquerque, New Mexico: Association for Computational Linguistics. <https://doi.org/10.18653/>

- v1/2025.findings-naacl.128
- Pustejovsky, J., Castaño, J., Ingria, R., Saurí, R., Gaizauskas, R., Setzer, A., ... Radev, D. R. (2003). TimeML: Robust Specification of Event and Temporal Expressions in Text. In *New Directions in Question Answering*. <https://cdn.aaai.org/Symposia/Spring/2003/SS-03-07/SS03-07-005.pdf>
- Rajaby Faghihi, H., Guo, Q., Uszok, A., Nafar, A., & Kordjamshidi, P. (2021a). DomiKnowS: A Library for Integration of Symbolic Domain Knowledge in Deep Learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 231–241). Online and Punta Cana, Dominican Republic: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-demo.27>
- Rajaby Faghihi, H., Guo, Q., Uszok, A., Nafar, A., & Kordjamshidi, P. (2021b). *DomiKnowS GitHub Repository*. GitHub. <https://github.com/HLR/DomiKnowS> (Architecture diagram accessed from README.md)
- Roccabruna, G., Rizzoli, M., & Riccardi, G. (2024). Will LLMs Replace the Encoder-Only Models in Temporal Relation Classification? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 20402–20415). Miami, Florida, USA: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.emnlp-main.1136>
- Su, X., Howard, P., & Bethard, S. (2025). Transformer-Based Temporal Information Extraction and Application: A Review. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 28822–28841). Suzhou, China: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.emnlp-main.1467>
- Sun, W. (2014). *Time will tell: Temporal reasoning in clinical narratives and beyond*. State University of New York at Albany.
- Tan, X., Pergola, G., & He, Y. (2021). Extracting Event Temporal Relations via Hyperbolic Geometry. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (pp. 8065–8077). Online and Punta Cana, Dominican Republic: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-main.636>
- Tatu, M., & Srikanth, M. (2008). Experiments with reasoning for temporal relations between events. In *Proceedings of the 22nd international conference on computational linguistics (COLING 2008)* (pp. 857–864). <https://aclanthology.org/C08-1108.pdf>
- Tourille, J., Ferret, O., Neveol, A., & Tannier, X. (2017). Neural Architecture for Temporal Relation Extraction: A Bi-LSTM Approach for Detecting Narrative Containers. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 224–230). Vancouver, Canada: Association for Computational Linguistics. <https://doi.org/10.18653/v1/P17-2035>

- UzZaman, N., Llorens, H., Derczynski, L., Allen, J., Verhagen, M., & Pustejovsky, J. (2013, June). SemEval-2013 Task 1: TempEval-3: Evaluating Time Expressions, Events, and Temporal Relations. In S. Manandhar & D. Yuret (Eds.), *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)* (pp. 1–9). Atlanta, Georgia, USA: Association for Computational Linguistics. <https://aclanthology.org/S13-2001/>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30. https://proceedings.neurips.cc/paper_files/paper/7181-attention-is
- Verhagen, M., Gaizauskas, R., Schilder, F., Hepple, M., Katz, G., & Pustejovsky, J. (2007). SemEval-2007 Task 15: TempEval Temporal Relation Identification. In E. Agirre, L. Màrquez, & R. Wicentowski (Eds.), *Proceedings of the Fourth International Workshop on Semantic Evaluations (SemEval-2007)* (pp. 75–80). Prague, Czech Republic: Association for Computational Linguistics. <https://aclanthology.org/S07-1014/>
- Verhagen, M., & Pustejovsky, J. (2008). Temporal processing with the TARSQI toolkit. In *COLING 2008: companion volume: demonstrations* (pp. 189–192). <https://aclanthology.org/C08-3012/>
- Verhagen, M., Saurí, R., Caselli, T., & Pustejovsky, J. (2010). SemEval-2010 Task 13: TempEval-2. In K. Erk & C. Strapparava (Eds.), *Proceedings of the 5th International Workshop on Semantic Evaluation* (pp. 57–62). Uppsala, Sweden: Association for Computational Linguistics. <https://aclanthology.org/S10-1010/>
- Wang, H., Chen, M., Zhang, H., & Roth, D. (2020). Joint Constrained Learning for Event-Event Relation Extraction. In B. Webber, T. Cohn, Y. He, & Y. Liu (Eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 696–706). Online: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.emnlp-main.51>
- Wang, L., Li, P., & Xu, S. (2022). DCT-Centered Temporal Relation Extraction. In N. Calzolari et al. (Eds.), *Proceedings of the 29th International Conference on Computational Linguistics* (pp. 2087–2097). Gyeongju, Republic of Korea: International Committee on Computational Linguistics. <https://aclanthology.org/2022.coling-1.182/>
- Xiong, S., Payani, A., Kompella, R., & Fekri, F. (2024). Large language models can learn temporal reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 10452–10470). <https://aclanthology.org/2024.acl-long.563.pdf>
- Yoshikawa, K., Riedel, S., Asahara, M., & Matsumoto, Y. (2009). Jointly identifying temporal relations with markov logic. In *Proceedings of the Joint Conference of the 47th Annual*

- Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP* (pp. 405–413). <https://aclanthology.org/P09-1046/>
- Yuan, C., Xie, Q., & Ananiadou, S. (2023). Zero-shot Temporal Relation Extraction with ChatGPT. In D. Demner-fushman, S. Ananiadou, & K. Cohen (Eds.), *Proceedings of the 22nd Workshop on Biomedical Natural Language Processing and BioNLP Shared Tasks* (pp. 92–102). Toronto, Canada: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.bionlp-1.7>
- Zhang, X., Zang, L., Liu, Q., Wei, S., & Hu, S. (2024). Event Temporal Relation Extraction based on Retrieval-Augmented on LLMs. In *2024 International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). Yokohama, Japan. <https://doi.org/10.1109/IJCNN60899.2024.10651084>
- Zhou, J., Dong, S., Tu, H., Wang, X., & Dou, Y. (2022). RSGT: Relational Structure Guided Temporal Relation Extraction. In N. Calzolari et al. (Eds.), *Proceedings of the 29th International Conference on Computational Linguistics* (pp. 2001–2010). Gyeongju, Republic of Korea: International Committee on Computational Linguistics. <https://aclanthology.org/2022.coling-1.174/>
- Zhou, L., & Hripcsak, G. (2007). Temporal reasoning with medical data—A review with emphasis on medical natural language processing. *Journal of Biomedical Informatics*, 40(2), 183–202. <https://doi.org/10.1016/j.jbi.2006.12.009>
- Zhou, Y., Yan, Y., Han, R., Caufield, J. H., Chang, K.-W., Sun, Y., ... Wang, W. (2021). Clinical Temporal Relation Extraction with Probabilistic Soft Logic Regularization and Global Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 35, pp. 14647–14655). <https://doi.org/10.1609/aaai.v35i16.17721>